

Foulkes' conjecture for the sixth symmetric power

OpenAI

September 25, 2026

Abstract

We prove the sixth-symmetric-power case of Foulkes' conjecture. For every integer $b \geq 6$ and every finite-dimensional complex vector space V , there is a $\mathrm{GL}(V)$ -equivariant injection $\mathrm{Sym}^6(\mathrm{Sym}^b V) \hookrightarrow \mathrm{Sym}^b(\mathrm{Sym}^6 V)$.

1 The result and its context

Foulkes' conjecture compares two ways of composing symmetric powers. For a finite-dimensional complex vector space V and integers $1 \leq a \leq b$, it asks for a $\mathrm{GL}(V)$ -equivariant injection

$$\mathrm{Sym}^a(\mathrm{Sym}^b V) \hookrightarrow \mathrm{Sym}^b(\mathrm{Sym}^a V).$$

Over $\mathbb{C}$, complete reducibility makes this equivalent to an inequality between the multiplicities of every irreducible polynomial representation. Write $\mathbf{S}_\lambda V$ for the Schur module indexed by a partition λ , and s_λ for its character. If h_r denotes the complete homogeneous symmetric function, then $h_a[h_b]$ is the stable character of $\mathrm{Sym}^a(\mathrm{Sym}^b V)$, where brackets denote plethysm. The conjecture says that $h_b[h_a] - h_a[h_b]$ is Schur-positive: all its Schur coefficients are nonnegative integers. We prove the sixth case.

Theorem 1.1. *For every integer $b \geq 6$ and every finite-dimensional complex vector space V , there is a $\mathrm{GL}(V)$ -equivariant injection*

$$\mathrm{Sym}^6(\mathrm{Sym}^b V) \hookrightarrow \mathrm{Sym}^b(\mathrm{Sym}^6 V).$$

Equivalently, $h_b[h_6] - h_6[h_b]$ is Schur-positive for every $b \geq 6$.

Theorem 1.1 resolves this case of Foulkes' conjecture positively, with no restriction on $\dim V$. The diagonal case $b = 6$ is immediate; the content is the comparison for every larger b .

History and the canonical map. The conjecture originates in Foulkes' study of concomitants [11]. For binary forms, classical Hermite reciprocity gives an isomorphism between the two symmetric powers for all a, b ; see [22, Introduction]. In higher dimension one seeks a multiplicity inequality instead. The case $a = 1$ is immediate. Thrall's decomposition formulas imply $a = 2$ [25]; see also [8, p. 237]. Dent and Siemons proved $a = 3$ [8].

A natural approach uses the canonical Foulkes–Howe map

$$\Psi_{a,b,V} : \mathrm{Sym}^a(\mathrm{Sym}^b V) \longrightarrow \mathrm{Sym}^b(\mathrm{Sym}^a V).$$

Realize the source as symmetric tensors in $V^{\otimes ab}$, arrange the tensor positions in a rows of length b , transpose the array, and symmetrize the resulting blocks. McKay proved that injectivity for every

V at a seed (a, b_0) , with $b_0 \geq a$, propagates to every (a, b) with $b \geq b_0$ [19, 14]. The computation at $(4, 4)$ by Müller and Neunhöffer therefore yields $a = 4$ [20]; Cheung, Ikenmeyer, and Mkrtchyan’s injectivity computation at $(5, 6)$ yields $a = 5$, with the diagonal comparison again immediate [7, preprint, Theorem 6(a)]. Canonical injectivity is stronger than Foulkes’ multiplicity comparison. The canonical maps at $(5, 5)$ and $(6, 6)$ have nonzero kernels in suitable dimensions, although their source and target representations coincide [20, 7]. For the sixth diagonal this already occurs when $\dim V \geq 6$ [7, preprint, Theorem 6(c)]. Our finite comparison does not require injectivity of a prescribed map.

Exact character calculations give another approach. Evseev, Paget, and Wildon verified Foulkes’ multiplicity inequalities when the two parameters sum to at most 19, including the sixth case through $b = 13$ [10, Corollary 5.2]. Their character-deflation recurrence is a predecessor of the exact source calculation used here. The present certificates combine such calculations with bounds on entire families of multiplicities, leaving only a small band for exact comparison.

Brion proved eventual truth for fixed a [2, Corollary 1.3] and later gave a dimension-dependent bound for surjectivity of the canonical map in the opposite direction [3, Theorem 3.3]. That map, $\Psi_{b,a,V}$, is realized as multiplication in the invariant ring associated with the normalization of the Chow variety [16, Section 7]; [22, Section 2]. The companion paper proves its surjectivity for $b \geq a(a - 1)$, uniformly in dimension [21, Theorem 1]. At $a = 6$, an invariant complement to its kernel supplies an embedding for $b \geq 30$. The remaining task is to compare all multiplicities for $6 \leq b \leq 29$.

For binary forms, recent generalized comparisons concern plethysms with different parameter pairs: Raicu, Sam, Weyman, and Yang prove maximal rank under divisibility conditions [23, Theorem 1.4], and Gangl, Gutiérrez, and Szwej identify the dual map through a substitution construction [13, Theorem 1.2]. These results concern a two-dimensional underlying space; the theorem above treats every finite dimension.

Proof strategy. The source $\text{Sym}^6(\text{Sym}^b V)$ has Schur support of length at most six. Let $F(\lambda)$ and $Q(\lambda)$ denote its multiplicity and the corresponding target multiplicity, respectively, for a partition λ of $6b$. Section 2 reduces the problem to these six-part partitions and supplies rectangular highest-weight shifts of known comparisons and a chart certificate for partitions with $\lambda_2 + \cdots + \lambda_6 \leq b$. The canonical multiplication map also proves the main large-degree range in Corollary 2.1.

The finite argument compares upper bounds for F with lower bounds for Q . Section 3 bounds source multiplicities by counting possible intermediate partitions in a signed strip recurrence, and gives exact recurrences for both characters. Section 4 amplifies known target multiplicities: a basis of highest-weight polynomials can be chosen with distinct leading monomials, so a sumset bound supplies many independent products of those polynomials. Multiplication by a single nonzero highest-weight polynomial then transfers these lower bounds to further weights. Section 5 organizes the resulting tests into a finite computation. It checks every partition for $6 \leq b \leq 25$. In increasing degrees from 26 onward, rectangular shifts reduce the remaining problem to $\lambda_6 < 6$; the certificates leave only partitions in the band $\lambda_3 + \cdots + \lambda_6 \leq 27$. Section 6 computes exact differences throughout this band by reusing seven truncated rational-function numerators. Section 7 proves the arithmetic bounds and assembles the range coverage.

An independent route is retained in full. The chart calculation used in the finite comparison, combined with polarization, proves canonical surjectivity for $b \geq 150$ in Proposition 2.7. The polarization argument is related to the successive weight shifts in the proof of McKay’s theorem [14, Section 5]. Both certificate programs run through $b = 149$, so the extended finite verification and this chart bound prove Theorem 1.1 without the quadratic-stabilization input. The main route needs

only the finite comparisons through $b = 29$. The complete programs are printed in the appendices and supplied with reference outputs and a reproduction script. The formulas explain what the programs compute; their exact execution supplies the finite nonnegativity assertions.

Throughout, we use characteristic-zero complete reducibility and highest-weight theory for polynomial GL-representations [12, Theorem 9.19 and Proposition 14.13], Schur functors and the Cauchy decomposition [12, Theorem 6.3 and Exercise 6.11(b)], and the Schur alternant formula [17, Chapter I, (3.1)]. Stable Schur coefficients are unchanged on specializing extra character variables to zero whenever the partition length remains admissible [17, Chapter I, (3.2)–(3.3)]. The character interpretation of plethysm is recalled in [17, Chapter I, Appendix A, Sections 7–8]. We also write e_r for the elementary symmetric function and set $h_0 = e_0 = 1$.

2 The large-degree range and finite reductions

The proof has a representation-theoretic reduction and a finite multiplicity comparison. We first specify the canonical map supplied by the companion and reduce the remaining comparison to partitions with at most six parts. A chart of the same invariant ring gives one of the finite certificates. The final subsection proves the independent large-degree bound used with the extended computation.

2.1 The multiplication map

For the moment let $a \geq 2$, and set

$$P = (\mathrm{Sym} V)^{\otimes a}, \quad W_j = P_{(j, \dots, j)}, \quad R_j = W_j^{\mathfrak{S}_a}, \quad R = \bigoplus_{j \geq 0} R_j.$$

The symmetric group permutes the tensor factors. We identify $\mathrm{Sym}^a W$ with $(W^{\otimes a})^{\mathfrak{S}_a}$ by the normalized average

$$w_1 \cdots w_a \mapsto \frac{1}{a!} \sum_{\sigma \in \mathfrak{S}_a} w_{\sigma(1)} \otimes \cdots \otimes w_{\sigma(a)}.$$

Its inverse is the restriction of the symmetric-product quotient. Let $A \subset R$ be the graded subalgebra generated by $R_1 = \mathrm{Sym}^a V$. For $v \in V$, put $z(v) = v^{\otimes a} \in R_1$. Multiplication defines the canonical Foulkes–Howe map

$$\mu_{a,b,V} : \mathrm{Sym}^b(\mathrm{Sym}^a V) \longrightarrow R_b = \mathrm{Sym}^a(\mathrm{Sym}^b V), \quad (1)$$

whose image is A_b . Thus $\mu_{a,b,V} = \Psi_{b,a,V}$ in the notation of the introduction. It is equivariant, so $A_b = R_b$ suffices for the desired embedding, by complete reducibility. This is the equal-multidegree invariant construction underlying the Chow normalization; see [22, Section 2]. We will use its explicit polynomial realization.

Corollary 2.1 (The main large-degree range). *For every finite-dimensional complex V and every $b \geq 30$, there is a GL(V)-equivariant injection $\mathrm{Sym}^6(\mathrm{Sym}^b V) \hookrightarrow \mathrm{Sym}^b(\mathrm{Sym}^6 V)$.*

Proof. The map in (1), with exactly the averaging convention above, is the canonical multiplication map of [21, Theorem 1]. That theorem gives $A_b = R_b$ for $a = 6$ and $b \geq 6(6 - 1) = 30$, including dimension zero. In positive dimension its kernel has an invariant complement, by complete reducibility of complex polynomial GL(V)-representations. Restriction to this complement is an isomorphism onto R_b ; its inverse gives an equivariant section and hence the stated injection. There is no assertion that this section is a prescribed canonical map. $\square$

2.2 Six variables and rectangular shifts

For the finite comparison take $a = 6$. If $b \geq 0$ and λ is a partition of $6b$, the Cauchy decomposition of $P = \text{Sym}(V \otimes \mathbb{C}^6)$ [12, Exercise 6.11(b)] gives the source multiplicity

$$F(\lambda) = \dim \left((\mathbf{S}_\lambda \mathbb{C}^6)_{(b, \dots, b)}^{\oplus 6} \right). \quad (2)$$

In particular it vanishes for partitions of length greater than six. Stable Schur coefficients allow us to work in $V = \mathbb{C}^6$: extra variables are specialized to zero, while longer target constituents already have nonnegative multiplicities. This covers every dimension.

Put $S = \text{Sym}(\text{Sym}^6 \mathbb{C}^6)$ with its ordinary symmetric-algebra grading, and let $Q(\lambda)$ be the multiplicity of $\mathbf{S}_\lambda \mathbb{C}^6$ in S_b . Pad every partition by zeros to six parts and write

$$d_i = \lambda_i - \lambda_{i+1}, \quad \lambda_7 = 0, \quad \lambda(d)_i = \sum_{j=i}^6 d_j, \quad w(d) = \sum_{i=1}^6 i d_i.$$

Also put $t_s(d) = \sum_{j>s} \lambda(d)_j$. We abbreviate $F(d) = F(\lambda(d))$ and $Q(d) = Q(\lambda(d))$ when $6 \mid w(d)$; then $b = w(d)/6$. Gap-vector inequalities are componentwise, and e_i denotes the i -th unit vector in gap coordinates.

Lemma 2.2. *Let $c, d \in \mathbb{Z}_{\geq 0}^6$ satisfy $6 \mid w(c)$ and $6 \mid w(d)$. If $Q(c) > 0$, then $Q(d + c) \geq Q(d)$. In particular*

$$Q(d + 6e_i) \geq Q(d) \quad (1 \leq i \leq 6), \quad F(d + 6e_6) = F(d).$$

Proof. Multiplication by one nonzero highest-weight polynomial of weight $\lambda(c)$ injects the highest-weight space at $\lambda(d)$ into that at their sum, because S is a domain. This is the usual highest-weight multiplication argument; compare [5, preprint, Lemma 2.2].

The rectangular occurrence $Q(6e_i) > 0$ is a special case of the even-partition theorem of Bürgisser, Christandl, and Ikenmeyer [4, Section 1.1, Theorem]. We give the determinant-tensor construction of [5, preprint, Corollary 4.8] in the form needed here. Take six copies of the determinant tensor on the first i basis vectors. Regroup the tensor positions into i blocks, each taking the same position from every copy, and symmetrize to $\text{Sym}^i(\text{Sym}^6 \mathbb{C}^6)$. This is a highest vector of weight $(6, \dots, 6)$ with i parts. Pair each block with the same sum of pure sixth powers of the first i dual basis vectors. A surviving term uses the same permutation in all six determinant copies, so its sign is $+1$; such terms exist. The vector is nonzero.

Adding $6e_6$ adds six to every partition part. In (2), this tensors with $\det^6$, shifts the equal label weight by six, and is trivial on permutation matrices. Thus $F(d + 6e_6) = F(d)$. $\square$

It follows that, after checking all partitions for $6 \leq b \leq 25$, induction in $26 \leq b \leq 29$ need only treat $d_6 < 6$. If $d_6 \geq 6$, subtract $6e_6$; the degree becomes $b - 6 \geq 20$, and Lemma 2.2 imports the earlier comparison. The same induction works throughout $26 \leq b \leq 149$, the full range retained in the programs for the independent route.

2.3 A chart certificate for the first row

We return to arbitrary $a \geq 2$ and V for the chart calculation. Its factor bound will show, at $a = 6$, that the multiplication image contains every source highest-weight space with $\lambda_1 \geq 5b$. It will also provide the independent large-degree bound in the next subsection. Dimension zero is immediate, so assume $V \neq 0$.

Fix $0 \neq v \in V$, use it as the first basis vector, and write $x_{i0}, x_{i1}, \dots$ for the corresponding polynomial variables in row i of P . Put $z = z(v) = \prod_i x_{i0}$. After inverting z , every x_{i0} is invertible. Write $Y_i = (x_{i1}/x_{i0}, x_{i2}/x_{i0}, \dots)$.

Lemma 2.3. *For every integer $t \geq 0$, every row-permutation-invariant polynomial in the lists Y_i of total degree at most t is a linear combination of products of at most t elements of $z^{-1}R_1$. Consequently, for integers $k, L \geq 0$ and $f \in R_k$,*

$$z^L f \in A_{k+L} \quad \text{if } k + L \geq ak.$$

Proof. The multisymmetric generators are described in [22, Remark 2.9]; we include the factor-length argument needed here. For the general multisymmetric generator theorems, see [26, Theorem 1] and [24, Corollary 8.4]. For each linear form ℓ in one list, the coefficients of

$$\frac{z(v + tu)}{z} = \prod_{i=1}^a (1 + t\ell(Y_i))$$

show that $e_h(\ell(Y_1), \dots, \ell(Y_a))$, and all their polarizations, lie in $z^{-1}R_1$, for $1 \leq h \leq a$. Here u ranges over the span of the remaining basis vectors.

Give e_h weight h . Newton identities express $\sum_i \ell(Y_i)^d$ as a polynomial in $e_1, \dots, e_a$ of weighted degree d , including when $d > a$. Each product has at most d factors. Polarizing in ℓ therefore expresses

$$p_m = \sum_i m(Y_i)$$

for every degree- d monomial m using at most d polarized elementary factors.

Invariant monomial orbit sums are scalar multiples of expressions

$$\sum_{\substack{i_1, \dots, i_h \\ \text{distinct}}} \prod_{\nu=1}^h m_\nu(Y_{i_\nu}),$$

where the m_ν have positive degree. Inclusion-exclusion over collisions of the indices expresses this in products of p_m 's; a collision replaces monomials by their product and preserves total degree. Thus total degree at most t requires at most t elementary factors. Constants require no factors.

For $f \in R_k$, the polynomial f/z^k is invariant and has total list degree at most ak . A term with $m \leq ak$ ratio factors has the form $z^{-m}g_1 \cdots g_m$, with $g_\nu \in R_1$. Multiplication by z^{k+L} gives an element of A_{k+L} when $k + L \geq ak$. Equality in the localization descends to P , a domain. $\square$

Corollary 2.4 (First-row certificate). *For every integer $b \geq 1$ and partition λ of $6b$ with $\sum_{i \geq 2} \lambda_i \leq b$, one has*

$$[s_\lambda]h_6[h_b] \leq [s_\lambda]h_b[h_6].$$

Proof. Partitions of length greater than six have zero source multiplicity, so work in $V = \mathbb{C}^6$ with its standard ordered basis and put $a = 6$. Choose v to be the first basis vector in the chart above. For every weight- λ polynomial $f \in R_b$, the total list degree of f/z^b is $t = \lambda_2 + \cdots + \lambda_6$. The first assertion of Lemma 2.3 writes this ratio as a linear combination of terms $z^{-m}g_1 \cdots g_m$, with $m \leq t \leq b$ and $g_\nu \in R_1$. Thus every corresponding term of f is

$$z^{b-m}g_1 \cdots g_m \in A_b.$$

The entire weight- λ space of R_b therefore lies in A_b . Since A_b is a $\text{GL}(V)$ -submodule of R_b , their highest-weight spaces at λ are equal: intersect the common weight space with the kernels of the positive simple-root operators. The surjection onto A_b induced by (1) then gives $Q(\lambda) \geq F(\lambda)$. $\square$

2.4 An independent large-degree bound

We now prove surjectivity of $\mu_{a,b,V}$ for $b \geq a(a-1)^2$, without using the companion. The chart will clear denominators uniformly once we know that R , as an A -module, is generated in degrees below a . Polarization supplies that finite-generation statement and then turns the chart calculation into surjectivity in large degree. We continue with arbitrary $a \geq 2$; the zero-dimensional case is immediate.

Lemma 2.5 (Polarization). *For integers $L \geq 1$ and $j \geq aL$,*

$$W_j = \sum_{v \in V} z(v)^L W_{j-L}, \quad R_j = \sum_{v \in V} z(v)^L R_{j-L}.$$

The sums mean linear spans.

Proof. The operator argument is related to the successive weight shifts in [14, Section 5]. Polarize the degree- aL polynomial $v \mapsto z(v)^L$, and multiply, to obtain a linear map

$$\mathrm{Sym}^{aL} V \otimes W_{j-L} \longrightarrow W_j.$$

Its image is the first displayed span. Identify W_j^* with polynomials $f(x_1, \dots, x_a)$, separately homogeneous of degree j . Introduce one auxiliary vector variable y . The dual map is, up to a nonzero scalar,

$$f \longmapsto \prod_{i=1}^a (y \cdot \partial_{x_i})^L f.$$

Indeed, evaluation of the dual pairing extracts the coefficient of $t_1^L \cdots t_a^L$ in $f(x_1 + t_1 y, \dots, x_a + t_a y)$.

On the polynomial space of total degree aj in $x_1, \dots, x_a, y$, the operators

$$D_i = y \cdot \partial_{x_i}, \quad E_i = x_i \cdot \partial_y, \quad H_i = \deg(x_i) - \deg(y)$$

form an $\mathfrak{sl}_2$, with D_i lowering weight by two. The other lists are spectator variables. The standard weight strings are described in [12, Section 11.1, (11.5)]. In an irreducible module of highest weight n , a weight $w \geq L$ is at distance $(n+w)/2 \geq L$ from the bottom. Therefore D_i^L is injective on the full weight- w space of any finite-dimensional module. After the first $i-1$ shifts, the relevant weight is $j - (i-1)L \geq L$. Every shift is injective on a space containing the preceding image, so their product is injective. Duality proves the first identity. Average its coefficients over $\mathfrak{S}_a$; each $z(v)^L$ is invariant, proving the second identity. $\square$

Corollary 2.6. *The A -module R is generated by $R_0, \dots, R_{a-1}$.*

Proof. Lemma 2.5 with $L = 1$ gives $R_j = R_1 R_{j-1}$ for $j \geq a$; apply induction on j . $\square$

Proposition 2.7 (Independent chart bound). *For $a \geq 2$, one has $R_b = A_b$ whenever $b \geq a(a-1)^2$.*

Proof. Take $L = (a-1)^2$, so that $L \geq (a-1)k$ for every $0 \leq k < a$. Lemma 2.3 therefore applies to each of these degrees. For any fixed nonzero v , multiply an A -linear expression in the generators of Corollary 2.6 by $z(v)^L$; this gives $z(v)^L R \subset A$. The same assertion is trivial for $v = 0$. If $b \geq aL$, Lemma 2.5 now yields

$$R_b = \sum_v z(v)^L R_{b-L} \subseteq A_b.$$

The opposite inclusion is definitional. $\square$

For $a = 6$, Proposition 2.7 supplies the independent large-degree range $b \geq 150$. Combined with the retained finite verification through $b = 149$, it proves Theorem 1.1 without the quadratic-stabilization input. The main proof needs only $6 \leq b \leq 29$, since Corollary 2.1 applies from $b = 30$.

3 An upper bound and two exact recurrences

We compare the source and target multiplicities without computing both at every partition. This section supplies a gap-monotone upper bound $U(d) \geq F(d)$, together with exact recurrences for the target tables and for the source values needed when that bound is inconclusive. All three constructions come from coefficient extraction with Schur alternants.

3.1 The signed strip rule

The coefficient rule below is the complete-symmetric case of the plethystic Murnaghan–Nakayama rule; see [9, pp. 28–29] and [27, arXiv:1408.3554v2, equation (2)]. Its residue-class determinant formulation also appears in [6, Theorem 3.4 and Corollary 3.11]. We include the proof and the containment inequalities in the form needed by the exact calculation.

For n variables put $\delta_n = (n-1, \dots, 0)$. The next rule bounds a single removal step in a product of factors $h_b(X^i)$, and also enumerates that step exactly.

Lemma 3.1 (Signed strips). *Let γ, μ be partitions of length at most n , and suppose $|\gamma| - |\mu| = ib$, where $i, b \geq 1$. Then*

$$[s_\gamma] h_b(X^i) s_\mu(X) \in \{0, 1, -1\}.$$

For a nonzero coefficient one necessarily has

$$\gamma_j \geq \mu_j, \quad \mu_j \geq \gamma_{j+i} \quad \text{whenever the subscripts are in range.}$$

Here $X^i = (x_1^i, \dots, x_n^i)$.

Proof. Set $l = \gamma + \delta_n$ and $a = \mu + \delta_n$. Multiplying by the Schur alternant denominator [17, Chapter I, equation (3.1)] gives

$$[s_\gamma] h_b(X^i) s_\mu(X) = \det [\mathbf{1}_{\{a_k \leq l_j, a_k \equiv l_j \pmod{i}\}}]_{j,k=1}^n.$$

Indeed its determinant expansion assigns the entries of a to positions j , with assigned values β_j satisfying

$$\beta_j \leq l_j, \quad \beta_j \equiv l_j \pmod{i}.$$

The size condition makes the sum of the decrement quotients equal to b , so there is no further constraint from $h_b(X^i)$.

Group the rows and columns by residue modulo i , preserving decreasing order within each group. Unequal row and column counts in a residue give zero, and empty blocks contribute one. In a square block of size m , each row support is a terminal interval of columns, and the supports decrease with the row index. A zero row or repeated support makes the determinant zero. Otherwise their sizes are $m, m-1, \dots, 1$: the block is upper triangular with diagonal entries one, and its only allowed matching is the diagonal. Translating this matching back to the original positions gives

$$\beta_j \leq l_j, \quad \beta_j > l_{j'} \quad \text{if } j' \text{ is the next position of the same residue.} \quad (3)$$

There is therefore just one surviving assignment. The sign from sorting the assembled β 's is the remaining row-and-column ordering sign, which proves the asserted coefficient values.

Sorting the componentwise inequalities $\beta_j \leq l_j$ gives $\mu_j \leq \gamma_j$. For the other inequality, consider the first $j + i$ entries of l . At most one per residue lacks a preceding entry of that residue. Each of the remaining at least j entries has a predecessor position whose assigned value, by (3), is at least $l_{j+i} + i$. The j -th largest assigned value is therefore at least $l_{j+i} + i$, which after removing the staircases says $\mu_j \geq \gamma_{j+i}$. $\square$

3.2 A monotone upper certificate

Averaging the permutations of six tensor factors projects onto $\text{Sym}^6(\text{Sym}^b V)$. Taking traces gives the cycle formula

$$h_6[h_b](X) = \frac{1}{720} \sum_{\pi \in \mathfrak{S}_6} \prod_{\text{cycles } c \text{ of } \pi} h_b(X^{|c|}). \quad (4)$$

We bound a coefficient of each product by removing its factors one at a time. The signed strip rule restricts the intermediate partitions; the bound below counts their possible coordinates while discarding their signs and some of their constraints.

For a nonnegative gap vector d , define

$$h_{sj} = 1 + \lambda(d)_j - \lambda(d)_{j+6-s}, \quad N_s(d) = \frac{\prod_{j=1}^s h_{sj}}{\max_{1 \leq j \leq s} h_{sj}} \quad (2 \leq s \leq 5), \quad N_0 = N_1 = 1.$$

For an ordered list $I = (i_1, \dots, i_v)$ with positive entries summing to six put

$$B_I(d) = \prod_{u=1}^v N_{6-i_1-\dots-i_u}(d), \quad U(d) = \left[\frac{1}{720} \sum_{\pi \in \mathfrak{S}_6} \min_{I \sim \pi} B_I(d) \right],$$

where $I \sim \pi$ means any ordering of the cycle lengths of π . This definition makes sense even when $6 \nmid w(d)$.

Proposition 3.2. *The function U is nondecreasing in every gap. For every nonnegative gap vector d with $6 \mid w(d)$, one has $F(d) \leq U(d)$.*

Proof. Each N_s is the minimum of the products obtained by omitting one of the widths h_{sj} . Every width is nondecreasing in the gaps, proving monotonicity through the products, minima, sum, and floor.

The zero vector has $F(0) = U(0) = 1$, so assume $b = w(d)/6 \geq 1$. Remove the factors of one product in (4) in an order I . After lengths totaling $6 - s$ have been removed, the remaining product is the trace of a permutation of the s factors on $(\text{Sym}^b V)^{\otimes s}$, together with the diagonal $\text{GL}(V)$ -action. The permutation commutes with that action, so its Schur support lies among the constituents of the tensor power. Those have length at most s by Pieri's rule [17, Chapter I, equation (5.16)]. Iterating Lemma 3.1, every possible intermediate partition μ satisfies

$$|\mu| = sb, \quad \lambda_{j+6-s} \leq \mu_j \leq \lambda_j \quad (1 \leq j \leq s).$$

Fix all but one coordinate; the sum fixes the remaining one. Thus there are at most $N_s(d)$ choices for $2 \leq s \leq 5$, and at most one for $s = 0, 1$. Each chain has absolute coefficient at most one. The coefficient of the product consequently has absolute value at most $B_I(d)$, for every order I . Average these bounds in (4) and use the integrality of $F(d)$. $\square$

In Appendix A, `init_masks` lists the eleven cycle types with their class sizes and the remaining lengths $s \in \{2, 3, 4, 5\}$ for each removal order. The routine `ub_cycle` forms the corresponding products of N_s , takes their minima, and performs the weighted sum and division by 720.

3.3 Exact coefficients

The target tables and the remaining source comparisons use Newton's identity in place of a bound. For an integer vector v of length n , a *signed lookup* is zero if v has a negative entry or a repeated entry. Otherwise sort v decreasingly, subtract δ_n , and multiply the coefficient at that partition by the sorting sign. A strictly decreasing nonnegative integer list is at least δ_n componentwise, so this subtraction always gives a partition.

Write $Q_b(\lambda) = [s_\lambda]h_b[h_6]$ and $\tilde{Q}_b(v)$ for its signed lookup in n variables. The initial value is $Q_0(0) = 1$. Newton's identity [17, Chapter I, equation (2.11)], multiplied by the alternating denominator, gives

$$bQ_b(\lambda) = \sum_{i=1}^b \sum_{\substack{\alpha \in \mathbb{Z}_{\geq 0}^n \\ |\alpha|=6}} \tilde{Q}_{b-i}(\lambda + \delta_n - i\alpha). \quad (5)$$

Indeed $h_6(X^i) = \sum_{|\alpha|=6} X^{i\alpha}$. Every nonzero previous partition μ in this formula satisfies $\mu \leq \lambda$ componentwise: subtracting a nonnegative vector cannot increase any decreasing order statistic, and the same staircase is then subtracted. In particular, a bound on a partition's tail is preserved in all previous lookups.

For the source, fix b and set

$$H_j(\gamma) = [s_\gamma]h_j[h_b], \quad H_0(0) = 1.$$

Newton's identity now gives the source recurrence; compare the character formulation in [10, Proposition 5.1]:

$$jH_j(\gamma) = \sum_{i=1}^j \sum_{\mu} [s_\gamma](h_b(X^i)s_\mu(X))H_{j-i}(\mu), \quad |\mu| = (j-i)b. \quad (6)$$

At stage j , $n = j$ variables suffice. Lemma 3.1 gives an economical enumeration. Put $l = \gamma + \delta_j$. In each residue modulo i , its last assigned value must be that residue's least nonnegative representative: the partition μ has length at most $j - i$, so its shifted list ends in $i - 1, \dots, 0$. If any residue is absent, there is no term. At every other position h , whose next position in its residue is h' , choose

$$\beta_h = l_h - ix_h, \quad 0 \leq x_h \leq (l_h - l_{h'})/i - 1.$$

The sum of all decrement quotients, including the fixed last ones, is b . These choices have distinct values: different residues cannot coincide, and in the same residue $\beta_h > l_{h'} \geq \beta_{h'}$. The fixed representatives $0, \dots, i - 1$ are the smallest i values. Thus sorting and subtracting δ_j gives a partition μ of length at most $j - i$, with the sorting sign. Every such choice contributes once. This is exactly the routine `strip` in Appendix A. Equation (5) is implemented by `gen`, `Make`, and `QTail`; (6) by `Source`.

4 A power bound from leading monomials

Rectangular shifts give a first lower bound for Q . The following bound amplifies a known multiplicity.

The sumset estimate is the repeated-set case of the Matolcsi–Ruzsa inequality [18, preprint, Theorem 1.5]; take $A = B = D$ and $r = k - 1$ there when $k \geq 2$. See also [1, author version, Corollary 2]. We include a self-contained triangulation proof. Its generic-point assignment is closely related to the half-open-simplex count used in [1, author version, Theorem 20], where the regions are instead obtained from a shelling.

Lemma 4.1 (Sumsets). *Let D be a nonempty finite set of C points in a real vector space, of affine dimension h . Write $kD = \{v_1 + \dots + v_k : v_1, \dots, v_k \in D\}$. For this k -fold sumset, $k \geq 1$, one has*

$$|kD| \geq f(C, h, k) := \binom{k+h}{h} + (C-1-h) \binom{k+h-1}{h}.$$

Proof. The case $h = 0$ is immediate. Triangulate $\text{conv } D$ using every point of D as a vertex and no other vertices. Such a triangulation can be built by starting with a full-dimensional simplex from D , then inserting points. For a point in the current hull, subdivide the simplices through its minimal containing face in the current triangulation by coning from it, retaining the induced common face subdivisions. For a point outside the hull, cone from it to the boundary simplices on strictly visible facets. Rays from the new point give the intersection and coverage assertions in the latter construction. Both operations preserve all previous vertices.

Choose p in the relative interior of the hull, off all hyperplanes of facets of full-dimensional simplices. Assign $x \in \text{conv } D$ to the unique full simplex whose interior contains $(1 - \epsilon)x + \epsilon p$ for all sufficiently small positive ϵ . There are finitely many facet hyperplanes, so this rule is well defined.

In barycentric coordinates of one full simplex, this assigned region requires nonnegative coordinates, with strict positivity exactly at the indices where the corresponding coordinate of p is negative. Assign a sum of k vertices according to its average, which lies in $\text{conv } D$. If there are e strict indices, the sums of k vertices of this simplex whose averages are assigned to it number $\binom{k-e+h}{h}$, interpreted as zero if $k < e$. Affine independence makes these points distinct; the assignment makes the counted sets for different simplices disjoint subsets of kD .

There is exactly one simplex with $e = 0$, namely the one containing p . At $k = 1$ the counting rule counts precisely the C vertices. To see this, a vertex of the complex lying in a full simplex must be a vertex of that simplex, by the common-face property. Its assigned simplex therefore counts it. A simplex with $e = 0$ contributes $h + 1$, one with $e = 1$ contributes 1, and all others contribute zero. Thus exactly $C - h - 1$ simplices have $e = 1$. Keeping just the $e = 0, 1$ contributions for arbitrary k proves the bound. $\square$

The leading-monomial argument in the next proposition is an elementary instance of the value-semigroup method: dimensions are counted by distinct values, and multiplication adds values; see [15, preprint, Propositions 2.3, 2.6, and 2.10]. We give the finite-degree argument and the dimension estimate explicitly, since their quantitative form is used in the certificate.

Proposition 4.2 (Power certificate). *Suppose $Q(x) \geq C > 0$, where C is an integer and $w(x) = 6d_0$ with $d_0 > 0$. Put*

$$z = \min \left(18, \min \left\{ h \geq 0 : \binom{d_0 + h}{h} \geq C \right\} \right).$$

Then for all $k \geq 1$,

$$Q(kx) \geq f(C, z, k). \tag{7}$$

The cap at 18 only weakens the lower bound; it limits the integer arithmetic in Section 7.

Proof. Choose C independent highest-weight polynomials at x in the polynomial algebra $S = \text{Sym}(\text{Sym}^6 \mathbb{C}^6)$. Row reduction with respect to a fixed monomial order gives a basis with distinct leading monomials. Their exponent vectors form a set D of size C , all with nonnegative integer entries summing to d_0 . For each point of kD , choose a product of k basis elements with that leading exponent. The products have distinct leading monomials and hence are independent highest-weight polynomials at kx . Thus $Q(kx) \geq |kD|$.

Let $h = \dim \text{aff } D$. Some projection onto h coordinate positions is injective on $\text{aff } D$, by a nonzero maximal minor of its direction space. The projected integer points are nonnegative with sum at most d_0 , so

$$C \leq \binom{d_0 + h}{h}, \quad z \leq h \leq C - 1.$$

Lemma 4.1 applies. Finally, for $0 \leq j < h$ one computes

$$\frac{f(C, j+1, k) - f(C, j, k)}{\binom{k+j-1}{j}} = \frac{(k-1)(C-j-1)}{j+1} \geq 0.$$

This proves $f(C, h, k) \geq f(C, z, k)$. The formula also covers $C = 1$, when $h = z = 0$, and $k = 1$. $\square$

5 Certificates in degrees below 150

We now combine the upper bound $F \leq U$, exact target multiplicities, and highest-weight multiplication. The base interval $6 \leq b \leq 25$ is checked directly. Beyond it, exact target values supply lower-bound seeds: they are computed from the target recurrence without assuming the desired comparison. The resulting certificates leave a set of partitions whose exact differences will be checked in Section 6. We retain the full interval through $b = 149$ for the independent chart proof; only $b \leq 29$ is needed with quadratic stabilization.

5.1 Initial tables and the base interval

Compute Q using (5) on the following four sets. All partitions have length at most six.

degrees	additional restriction	variables used
$0 \leq b \leq 25$	none	6
$0 \leq b \leq 149$	$\text{length}(\lambda) \leq 3$	3
$0 \leq b \leq 55$	$\text{length}(\lambda) \leq 4$	4
$0 \leq b \leq 45$	$t_3(d) \leq 18$	6

(8)

Previous-degree lookups remain in the applicable set by Section 3. The first table supplies the direct base comparisons; all four supply lower-bound seeds for larger degrees. Overlaps are immaterial.

For $6 \leq b \leq 25$, every partition is first tested against $Q(d) \geq U(d)$. When this fails, compute $F(d) = H_6(\lambda(d))$ exactly by (6). Only these requested values at the last stage $j = 6$ are needed. Table 1 records all the fallback comparisons and their equalities.

5.2 Residues and transport of lower bounds

For $26 \leq b \leq 149$ we need only consider $d_6 < 6$, by Lemma 2.2. There is a unique expression

$$d = r + 6m, \quad r \in \mathcal{R} = \{0, \dots, 5\}^6 \cap \{6 \mid w(r)\}, \quad m \in \mathcal{G} = \{m \in \mathbb{Z}_{\geq 0}^6 : m_6 = 0, w(m) \leq 149\}.$$

b	exact comparisons	equalities	b	exact comparisons	equalities
6	2428	2428	16	7370	354
7	4476	315	17	7599	369
8	6820	255	18	7801	384
9	6899	250	19	8057	399
10	6886	264	20	8233	414
11	6814	279	21	8427	429
12	6850	294	22	8650	444
13	6872	309	23	8850	459
14	6973	324	24	9031	474
15	7131	339	25	9268	489

Table 1: The base interval: 145435 exact fallback comparisons, 9272 equalities, and no negative difference. All other partitions pass $Q \geq U$.

Write $b_0(r) = w(r)/6$, so that $b = b_0(r) + w(m)$. There are 7776 residues: choose $r_2, \dots, r_6$, and r_1 is uniquely determined modulo six. Moreover $b_0(r) \leq 17$. The grid $\mathcal{G}$ has 6611697 points. We call its indices m modes: fixing m groups the 7776 possible residues for simultaneous testing. Pairs whose degree lies outside $26 \leq b \leq 149$ will not require a test.

For an array on a downward-closed set, *propagation* replaces its value at m by the maximum of all values at indices $u \leq m$. Extending an array means filling new positions by zero and propagating. An increasing- w sweep taking maxima with the five immediate predecessors performs this operation. A reverse sweep through the same predecessors forms the downward closure of a set. Lemma 2.2 shows that propagation preserves every lower bound for $Q(r + 6m)$, with r fixed.

The same multiplication lemma also transports a multiplicity from residue zero to residue r . Suppose $J(u) \leq Q(6u)$ for $u \in \mathcal{G}$, $M(m) \leq Q(r + 6m)$, and $E \subseteq \mathcal{G}$ is a set of indices satisfying $Q(r + 6c) > 0$ for $c \in E$. Multiplication by one nonzero highest-weight polynomial at $r + 6c$ gives

$$Q(r + 6m) \geq \max(\{M(m)\} \cup \{J(m - c) : c \in E, c \leq m\}). \quad (9)$$

The right side certifies the comparison whenever it is at least $U(r + 6m)$. No product of two multiplicity dimensions is used: a single nonzero factor gives each injection.

We first construct J and choices of M, E valid for every residue, so that one test can discard a whole mode. For the modes left over we use more precise choices depending on r . The following construction specifies all arrays and tests; the common cap $Z = 2^{100}$ is applied only to lower bounds.

5.3 Certificates shared by all residues

A shared power table. Initialize $J(m) = 1$ on $\mathcal{G}$, using the rectangle occurrences and the nonzero constant at $m = 0$. For every gap vector x with $x_6 = 0$, $Q(x) > 1$, and known value in one of the first three sets of (8), put $d_0 = w(x)/6$ and $C = \min(Q(x), Z)$. For each integer $k \geq 1$ with $kd_0 \leq 149$ and all entries of kx divisible by six, update

$$J(kx/6) \leftarrow \max\{J(kx/6), \min(Z, f(C, z, k))\},$$

where z is as in Proposition 4.2. Then propagate. It follows that

$$J(m) \leq Q(6m) \quad (m \in \mathcal{G}). \quad (10)$$

Common multiplicities and occurrences. For each r , initialize the exact array $Q(r + 6u)$ on $w(u) \leq 25 - b_0(r)$, then extend it to $w(m) \leq 42$. Take the pointwise minimum of these propagated arrays over all r , with a cap at Z , to obtain M_0 on that grid. Let E_0 be the minimal points where $M_0 > 0$, before extending M_0 to all of $\mathcal{G}$. Then

$$M_0(m) \leq Q(r + 6m) \quad \text{for every } r, \quad Q(r + 6c) > 0 \quad \text{for every } r \text{ and } c \in E_0. \quad (11)$$

The exact entries supplying the minimum may come from different predecessors for different r . Their common consequence is precisely the two universal inequalities in (11).

Selecting modes. Keep a mode $m \in \mathcal{G}$ unless either $w(m) + 17 \leq 25$, or

$$\max(\{M_0(m)\} \cup \{J(m - c) : c \in E_0, c \leq m\}) \geq U(6m + (5, 5, 5, 5, 5, 5)). \quad (12)$$

The first condition puts every residue in the already settled base range whenever its degree is at least six. For the second, use (9) with $M = M_0$ and $E = E_0$. Proposition 3.2 bounds $F(r + 6m)$ by $U(6m + (5, 5, 5, 5, 5, 5))$, because $r \leq (5, 5, 5, 5, 5, 5)$. The high-residue gap vector need not be admissible; U is defined and monotone there nonetheless.

5.4 Tests for each remaining residue

Form the downward closure of the retained modes. On this closure, initialize $Q(r + 6m)$ wherever it is available in (8), use zero elsewhere, and propagate to obtain M_r . Let E_r be the minimal positive locations of the exact array $Q(r + 6u)$ on $w(u) \leq 25 - b_0(r)$. For each originally retained mode with $26 \leq b_0(r) + w(m) \leq 149$, put $d = r + 6m$ and clear the pair if either

$$t_1(d) \leq b \quad \text{or} \quad \max(\{M_r(m)\} \cup \{J(m - c) : c \in E_r, c \leq m\}) \geq U(d).$$

Flag it otherwise. The first test is Corollary 2.4. The second is (9) with $M = M_r$ and $E = E_r$.

Minimal positive locations can be found by checking the immediate predecessors. Indeed positive exact multiplicities form an upper set within the small grid, by the rectangular shifts; the propagated common array has the same property. The set used for M_r retains every predecessor needed for propagation. Each difference $m - c$ in a mixed test is nonnegative and lies in $\mathcal{G}$.

In Appendix A, `powerAll` stores J , `uniBase` and `u0cc` store M_0 and E_0 . The routine `setup` retains the modes and their downward closure, `init_occ` constructs E_r , and `TestSpec` propagates M_r and tests the remaining pairs. The routine `run` performs the base-interval comparisons.

Finite verification 5.1. *The complete program in Appendix A verifies the base comparisons in Table 1. Its mode list has 87354 points, or 114314 after taking the downward closure. It flags 4969698 pairs in the middle interval, all with $t_2(d) \leq 27$. Their counts by degree range appear in Table 2.*

Every pair in $26 \leq b \leq 149$ cleared by these tests satisfies $Q \geq F$, by the proved upper and lower bounds or the chart. Verification 5.1 supplies the separate finite assertion that every pair left over has $t_2 \leq 27$. Thus it remains only to check the entire band $t_2 \leq 27$ in $26 \leq b \leq 149$.

6 Exact differences in the small-tail band

We compute the entire band

$$\begin{aligned} 26 \leq b \leq 149, \quad \lambda = (6b - B - |\eta|, B, \eta), \\ \eta_1 \geq \cdots \geq \eta_4 \geq 0, \quad |\eta| \leq 27, \quad \eta_1 \leq B \leq \left\lfloor \frac{6b - |\eta|}{2} \right\rfloor. \end{aligned}$$

In particular it contains every flagged partition of Verification 5.1. For each fixed tail η , we will construct seven Laurent series independent of b . Their coefficients at $B, B - b, \dots, B - 6b$ will sum to $720(Q(\lambda) - F(\lambda))$. The same seven rows therefore serve every degree and every second part in the displayed range. Write $\mathcal{Q}_b = h_b[h_6]$, $\mathcal{F}_b = h_6[h_b]$, and set

$$\mathcal{C}(q, y) = (1 - q) \prod_{\ell=1}^4 (1 - y_\ell)(1 - y_\ell/q), \quad \mathcal{A}(y) = \prod_{\ell < h} (1 - y_h/y_\ell), \quad D(q) = \prod_{h=1}^6 (1 - q^h).$$

We first expand the characters and geometric products in the tail variables y , and then expand their rational coefficients in q as Laurent series at $q = 0$. In particular, $D(q)^{-1}$ has only nonnegative powers.

6.1 Coefficient extraction and truncation

The normalized alternating Schur formula gives

$$Q(\lambda) - F(\lambda) = [q^B y^\eta] \mathcal{C}(q, y) \mathcal{A}(y) (\mathcal{Q}_b - \mathcal{F}_b)(1, q, y). \quad (13)$$

Indeed in six variables it extracts the coefficient of x^λ after multiplication by $\prod_{i < j} (1 - x_j/x_i)$. Distinct decreasing shifted partitions cannot be permutations of each other, so only the Schur term at λ contributes. All Laurent monomials have total degree $6b$; setting the first variable to one loses no information about its exponent once the remaining five are fixed.

Give each y_ℓ degree one and q degree zero. The monomials of $\mathcal{A}$ have total y -degree zero; those of $\mathcal{C}$ have nonnegative total y -degree. Character terms above degree 27 therefore cannot contribute to (13). This uses total degree even when individual exponents in the alternant are negative. Write $\equiv_{27}$ for equality of coefficients of nonnegative y -monomials through total degree 27, with coefficients in $\mathbb{Q}(q)$.

6.2 Seven numerators independent of the degree

For $0 \leq j \leq 6$ define

$$L_j(q, y) = \prod_{\substack{\tau \in \mathbb{Z}_{\geq 0}^4, \ 1 \leq |\tau| \leq 6 \\ 0 \leq u \leq 6 - |\tau|}} (1 - y^\tau q^{u-j})^{-1}.$$

Then

$$\mathcal{Q}_b(1, q, y) \equiv_{27} \frac{1}{D(q)} \sum_{j=0}^6 q^{jb} (-1)^j e_j(q, q^2, \dots, q^6) L_j(q, y). \quad (14)$$

To prove this, fix a multiset of k inner monomials with nonzero tail. The remaining $b - k$ entries come from $1, q, \dots, q^6$, with generating polynomial

$$h_{b-k}(1, q, \dots, q^6) = \prod_{h=1}^6 \frac{1 - q^{b-k+h}}{1 - q^h} \quad (b - k \geq 0).$$

This Gaussian polynomial counts partitions in a 6-by- $(b - k)$ rectangle. In general their generating polynomial $P_{r,s}$, with at most r rows of length at most s , satisfies $P_{r,s} = P_{r,s-1} + q^s P_{r-1,s}$, by removing a row of length s when one is present, and $P_{0,s} = P_{r,0} = 1$. The displayed product with $r = b - k, s = 6$ satisfies the same recurrence and boundary values, proving the formula. Expanding the numerator gives the e_j in (14); the factor q^{-jk} contributes q^{-j} per selected tail monomial, and

hence produces L_j . Within the cutoff $k \leq 27$ and $b \geq 26$, so the only possible negative value of $b - k$ is -1 . Its numerator has the factor $1 - q^0$ and vanishes, exactly as required. Thus the formula includes the boundary $b = 26, k = 27$; its cancellation is across the seven j -terms.

Similarly, the binary complement of one tail monomial gives

$$h_b(1, q, y) \equiv_{27} \frac{\prod_{\ell}(1 - y_{\ell})^{-1} - q^{b+1} \prod_{\ell}(1 - y_{\ell}/q)^{-1}}{1 - q}. \quad (15)$$

At tail degree t , its coefficient is $(1 - q^{b+1-t})/(1 - q)$. It counts the binary monomials when $t \leq b$ and is zero at $t = b + 1$, the only extra possible degree.

For a cycle type $I = (i_1, \dots, i_v) \vdash 6$, let

$$n_I = \frac{720}{\prod_{r \geq 1} r^{m_r} m_r!}, \quad E_I(q) = \frac{D(q)}{\prod_{h=1}^v (1 - q^{i_h})},$$

where m_r counts cycles of length r . The quotient E_I is an integer polynomial of degree 15: the multiplicity of the cyclotomic factor Φ_d in its denominator is the number of cycle lengths divisible by d , which is at most $\lfloor 6/d \rfloor$, its multiplicity in D . Apply (15) with variables raised to each cycle length in (4). Together with (14) this yields

$$720D(q)(\mathcal{Q}_b - \mathcal{F}_b)(1, q, y) \equiv_{27} \sum_{j=0}^6 q^{jb} G_j(q, y), \quad (16)$$

where the seven numerators are explicitly

$$\begin{aligned} G_j(q, y) = & 720(-1)^j e_j(q, \dots, q^6) L_j(q, y) \\ & - \sum_{I \vdash 6} n_I \sum_{\substack{J \subseteq \{1, \dots, v\} \\ \sum_{h \in J} i_h = j}} (-1)^{|J|} q^j E_I(q) \prod_{\ell=1}^4 \prod_{h=1}^v (1 - y_{\ell}^{i_h} q^{-i_h \mathbf{1}_{h \in J}})^{-1}. \end{aligned} \quad (17)$$

Here $\mathbf{1}_{h \in J}$ is the indicator of membership. Every selected second term of (15) contributes $-q^{i_h(b+1)}$, explaining the sign and the residual q^j after q^{jb} is factored out. Equal-length cycles still have distinct positions in the subset sum. These formulas involve eleven cycle types and seven values of j , regardless of b .

6.3 Computing the numerators

Write $L_{j,a} = [y^a] L_j$. Euler differentiation of the product gives

$$L_{j,0} = 1, \quad |a| L_{j,a} = \sum_{\tau, u} \sum_{\substack{i \geq 1 \\ i\tau \leq a}} |\tau| q^{i(u-j)} L_{j,a-i\tau} \quad (|a| > 0). \quad (18)$$

For example, this follows by differentiating the logarithmic series $\sum_{i \geq 1} (y^{\tau} q^{u-j})^i / i$. Its denominator i cancels the extra i from differentiation. Every lookup has smaller total tail degree, and the division by $|a|$ is exact.

For the source product in (17), put

$$A_J(m) = [z^m] \prod_{h \in J} (1 - z^{i_h})^{-1}.$$

At a single tail-coordinate exponent m , its Laurent coefficient is

$$\sum_{p=0}^m A_J(p) A_{\bar{J}}(m-p) q^{-p}.$$

Multiply these four one-coordinate polynomials and then $(-1)^{|J|} n_I q^j E_I(q)$. These are the **ways** arrays and convolutions of **source** in Appendix B. The routines **target** and **gen_source** implement (18) and the cycle-type enumeration.

All raw arrays are symmetric in the four tail variables, so they need only be stored at decreasing nonnegative exponent vectors; arbitrary orders are looked up by sorting, without a sign. To multiply by the cross factors in $\mathcal{C}$, the coefficient at a shift $v \in \{0, 1, 2\}^4$ is

$$(-1)^{n_1} q^{-n_2} (1 + q^{-1})^{n_1}, \quad n_s = \#\{\ell : v_\ell = s\}.$$

An in-place sweep in decreasing tail degree reads unmodified lower-degree rows. A subsequent decreasing-exponent sweep multiplies by $1 - q$. This is the routine **cross**.

The cross factors preserve symmetry in the four tail variables. To extract a Schur coefficient, we now apply the alternating factor $\mathcal{A}$; the permutation signs enter at this step, rather than in the sorted lookups. Put $\rho = (3, 2, 1, 0)$, and let $M_{j,a} = [y^a] \mathcal{C} G_j$, with value zero at a vector with a negative coordinate. The exact row produced by **schur_row** is

$$R_{j,\eta}(q) = \frac{1}{D(q)} \sum_{\pi \in \mathfrak{S}_4} \text{sgn}(\pi) M_{j,\eta+\rho-\pi\rho}(q). \quad (19)$$

This follows from $\mathcal{A}(y) = y^{-\rho} \sum_{\pi} \text{sgn}(\pi) y^{\pi\rho}$. Every accepted index in (19) has nonnegative entries and total degree $|\eta| \leq 27$; in particular every coordinate is at most 27. There are 1908 decreasing four-coordinate tails in this range. Equations (13) and (16) show that the final integer tested is precisely

$$\sum_{j=0}^6 [q^{B-jb}] R_{j,\eta}(q) = 720 (Q(6b - B - |\eta|, B, \eta) - F(6b - B - |\eta|, B, \eta)). \quad (20)$$

The notation on the right specifies partitions rather than gap vectors.

6.4 The Laurent window

At total tail degree t , the coefficients and individual Euler summands of L_j have q -support in $[-6t, 5t]$. The elementary factor adds at most 21. A cross term consuming s tail units shifts down by at most s , while its raw input has degree $t - s$; the factor $1 - q$ adds at most one. Thus all target numerator summands lie in

$$[-6t, 5t + 22].$$

The source has raw support in $[-t, 21]$ and final support in $[-t, 22]$, using $\deg E_I = 15$ and $0 \leq j \leq 6$. Alternation does not change total tail degree or q -support. At $t \leq 27$ all these supports are therefore contained in

$$[-162, 157] \subset [-168, 161],$$

the interval stored by the program. Its clipping operation discards only zero terms. In particular all terms of the boundary cancellation at $b = 26, k = 27$ are retained separately.

Since $D(0) = 1$, its inverse is a power series with no negative exponents. Starting from exponent -168 , six forward passes $a_n \leftarrow a_n + a_{n-h}$, for $h = 1, \dots, 6$, divide by D exactly. The numerator is zero above 161. Expansion through 447 suffices because

$$B - jb \leq B \leq 3b \leq 447.$$

Entries below -168 are zero; every accessed nonnegative array index is at most $168 + 447 = 615$, within the 616 allocated entries. Neither division nor the shift q^{jb} can bring a discarded high exponent down to a requested one. The program converts to arbitrary-precision integers before the denominator passes and uses them for all final sums and divisibility tests.

Finally, `run_thin` visits each of the 1908 tails once, constructs its seven rows, and loops over the displayed ranges of b and B . For a fixed b, η , the number of permitted second parts is

$$3b - \left\lceil \frac{|\eta|}{2} \right\rceil - \eta_1 + 1.$$

This is positive throughout the band. Summing it over the tails and $26 \leq b \leq 149$ gives 57065668 distinct partitions; there is no dependence on which partitions the first program flagged.

Finite verification 6.1. *The complete program in Appendix B checks 57065668 coefficients in this band. Of these, 177134 are zero and none are negative; every integer in (20) is divisible by 720. The grouped counts appear in Table 2.*

7 Finite arithmetic, reproduction, and conclusion

7.1 Indexing and arithmetic in the certificate program

Appendix A lists the complete program used for Verification 5.1; it has no external data input. Its partition order reads parts from last to first. If $p_j(u)$ counts partitions of u into at most j parts, the prefix table counts those whose last padded part is $< k$, by

$$N_j(u, k) = N_j(u, k-1) + p_{j-1}(u - j(k-1)).$$

The second summand subtracts the last part from all j positions. Negative arguments count as zero and $p_0(0) = 1$. Subtracting two prefix counts ranks the next part after the already fixed smaller neighbor. A shorter partition padded by zeros has the same rank. For the tail-restricted table, first group by its exact last three parts, subtract their largest part from the preceding three, and use the three-part rank. The grid similarly groups by $w(m)$ and ranks the partition corresponding to its first five gaps.

The allocated prefix array has size parameter 1250. Its terminal weight need not supply a complete unrestricted count; all counts used for multiplicities and grid indices have weight at most 894, strictly below that terminal boundary. The prefix entries are bounded by $\binom{1254}{5} < 2^{63}$. The actual largest complete levels used in six, four, and three variables have respectively 1229120, 261072, and 67051 partitions. The largest tail-restricted level has 1263138 entries, and the grid has 6611697. These ranks, offsets, and predecessor indices fit signed 32-bit integers. Independently of these evaluated counts, coarse bounds for the three complete levels, the grid, and any tail level are

$$\max \left\{ \binom{155}{5}, \binom{333}{3}, \binom{896}{2}, \binom{154}{5}, \binom{21}{3} \binom{272}{2} \right\} < 2^{31}.$$

These count unrestricted weak compositions. For the grid, $w(m) \leq 149$ implies $\sum_i m_i \leq 149$; for a tail level there are at most $\binom{21}{3}$ ordered tails of size at most 18, and each remaining three-part prefix has weight at most 270.

Multiplicity arithmetic is signed 128-bit. The target recurrences check every accumulation for overflow, and test integrality and nonnegativity before division. For the source, only $b \leq 25$ is needed. Each intermediate multiplicity is at most 151^{10} : it is bounded by its multiplicity in $(\text{Sym}^b V)^{\otimes j}$, $j \leq 6$, and the successive interlacing partitions have at most $151^{0+1+2+3+4}$ choices. In one source Newton step, there are fewer than $6 \cdot 151^4 + 6$ strip terms, by fixing one coordinate with the size constraint. Thus every signed accumulation is bounded absolutely by

$$151^{10}(6 \cdot 151^4 + 6) < 2^{127}.$$

For U , every width in every actual or high-residue call is at most 920. A chain has at most $1 + 2 + 3 + 4 = 10$ width factors, so

$$B_I \leq 920^{10} < Z = 2^{100}.$$

The minimum initialized at Z therefore never clips an upper bound, and summing class weights costs a factor at most 720.

For the power bound, S has $\dim \text{Sym}^6 \mathbb{C}^6 = \binom{11}{5} = 462$ polynomial generators. Consequently $\binom{d_0+461}{461} \geq Q(x)$, so the binomial search for z , before its cap at 18, stops by 461. With $C \leq Z$ and $d_0 \leq 149$, its first crossing is at most $150Z$; predivision multiplication costs at most another factor 610, still below 2^{127} . After the cap, the other binomials are at most $\binom{167}{18}$. Products in the final lower bound are screened against Z before multiplication. Capping can only weaken a lower certificate. These observations cover the unchecked arithmetic as well as the program's explicit assertions.

OpenMP distributes distinct coefficient destinations within one degree. All previous degrees are read-only, and each work-sharing loop has its barrier before the next degree. The mathematical result is therefore independent of the thread count.

7.2 Integer bounds for the band program

It remains to bound the signed 128-bit arithmetic before the arbitrary-precision division described in Section 6. For a Laurent series truncated to total tail degree at most 27, use the sum of absolute coefficients as its norm. The norm of L_j is bounded by

$$N = \sum_{n=0}^{27} [z^n] P(z), \quad P(z) = \prod_{h=1}^6 (1 - z^h)^{-M_h}, \quad M_h = (7-h) \binom{h+3}{3}.$$

Here $(M_1, \dots, M_6) = (24, 50, 80, 105, 112, 84)$. There is a convenient purely elementary estimate

$$N \leq 4^{27} P(1/4) < 2^{79}. \tag{21}$$

The first inequality follows from nonnegative coefficients. For the second, put $r_h = 4^h/2$. Bernoulli's inequality gives $(1 - 4^{-h})^{r_h} > 1/2$. Thus the h -th factor of $P(1/4)$ is at most $2^{\lceil M_h/r_h \rceil}$, with strict total inequality. The six ceilings are 12, 7, 3, 1, 1, 1, whose sum is 25; combine this with $4^{27} = 2^{54}$.

Each four-coordinate source product has at most 24 geometric factors, each consuming at least one tail unit, so its norm is at most

$$S_0 = \binom{51}{24} < 2^{51}.$$

Using $\|D\|_1 \leq 64$, the quotient E_I has norm at most

$$Q_0 = 64 \binom{21}{6} = 3472896 < 2^{22}.$$

Indeed dominate all reciprocal cycle factors by $(1 - q)^{-6}$ and sum through degree 15. The partial quotient calculations through degree 21 have bound $64 \binom{27}{6} = 18944640$, and the one-coordinate arrays A_J have coefficients at most $\binom{32}{5} = 201376$. Their additions fit signed 32-bit integers. Class-size divisions are exact: for each partial cycle multiset of size $r \leq 6$, its centralizer order divides $r!$, hence divides 720.

The undivided Euler step costs at most $27N$. The absolute weight of all target elementary masks is $720 \cdot 64$, and the class-subset weight is

$$\sum_{I \vdash 6} n_I 2^{v(I)} = \sum_{\sigma \in \mathfrak{S}_6} 2^{\#\text{cycles}(\sigma)} = 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 = 5040.$$

The last identity follows inductively by inserting the new largest element into an existing cycle or making it a new cycle. Cross multiplication including $1 - q$ has norm at most 512; alternation costs at most 24. A bound for every prevision coefficient, partial sum, and product is therefore

$$512 \cdot 24(27 \cdot 720 \cdot 64N + 5040Q_0S_0) < 2^{115} < 2^{127}. \quad (22)$$

The source convolutions have nonnegative factors before their final signs, so this also bounds their intermediate products. Potentially large products in the code already have a 128-bit operand. The row divisions and final sums then use arbitrary precision. Each output counter covers at most six degrees, 1908 tails, and 448 second parts per degree and tail. Thus even the coarse bound $1908 \cdot 6 \cdot 448 < 2^{31}$ keeps these counters within signed 32-bit range.

7.3 Recorded computations and reproduction

The reference output streams contain 20 base rows, 21 flag rows, and 21 band rows. Tables 1 and 2 present all 62 numeric rows, including every tested degree. The reproduction script checks these streams against fixed hashes and compares their numeric contents with both manuscript tables.

A full reproduction compiled the two printed sources with GNU C++ 13.3.0 and executed both programs over their entire stated ranges, with assertions enabled. The first used two OpenMP threads and the second was serial. Both exited with status zero, and all four output streams matched the reference streams byte for byte. This execution reproduced all 62 numeric rows in the tables.

The programs retain the complete ranges $6 \leq b \leq 149$. For the main proof using Corollary 2.1, only the base comparisons $6 \leq b \leq 25$ and the first row of Table 2, for $26 \leq b \leq 29$, are required. That row contains 29814 flags and 467068 band coefficients; the band differences are nonnegative, with 2114 equalities. The later rows give the additional finite checks for the independent route through Proposition 2.7. Exact target tables at higher degrees are computed from the recurrences, not assumed from Foulkes' conjecture, so their use in lower certificates introduces no circularity.

The supporting package contains the complete program sources, reference output streams, their hashes and numeric tables, and a reproduction script. A new full run creates its own compiler and execution metadata. The commands, run from the paper directory, are

```
python3 -B verification/verify_computations.py --check
python3 -B verification/verify_computations.py --run --threads 2
```

b range	flagged pairs	largest t_2	band coefficients	zero differences
26–29	29814	25	467068	2114
30–35	46805	25	872322	3621
36–41	50091	25	1078386	4161
42–47	54443	25	1284450	4701
48–53	63027	25	1490514	5241
54–59	74729	20	1696578	5781
60–65	90684	18	1902642	6321
66–71	109505	18	2108706	6861
72–77	130920	19	2314770	7401
78–83	155274	20	2520834	7941
84–89	182871	21	2726898	8481
90–95	214444	22	2932962	9021
96–101	248073	23	3139026	9561
102–107	285382	24	3345090	10101
108–113	324614	25	3551154	10641
114–119	366623	26	3757218	11181
120–125	410723	26	3963282	11721
126–131	457715	26	4169346	12261
132–137	506088	26	4375410	12801
138–143	557588	26	4581474	13341
144–149	610285	27	4787538	13881
total	4969698	27	57065668	177134

Table 2: Full retained verification range. The first row is the residual range required with quadratic stabilization; subsequent rows support the independent chart route. All flags lie in the checked band, and no negative difference is found. The program’s first row is labeled 24, the multiple of six below the first tested degree; the explicit degree ranges here avoid that convention.

The first checks source and reference-stream identities, all 62 numeric rows, agreement with the manuscript tables, and the numerical bounds above; it does not execute the mathematical programs. The second compiles both unchanged sources with `-std=c++17 -O3 -UNDEBUG`, enables OpenMP only for the first, executes them, and requires both complete output streams to match the reference streams byte for byte. It keeps each run in a new external directory, with the compiler version, build and execution commands, exit statuses, timings, hashes, and captured streams. The option `--serial` may replace `--threads 2` to compile and run the first program without OpenMP. The two C++ files are also printed in full below, so every finite step is specified in this paper. The coefficient formulas, indexing arguments, and arithmetic bounds establish what they check; successful execution supplies the finite nonnegativity assertions.

7.4 Completion of the proof

Proof of Theorem 1.1. Corollary 2.1 settles $b \geq 30$. Verification 5.1 settles $6 \leq b \leq 25$. Proceed by increasing b from 26 through 29. If $d_6 \geq 6$, Lemma 2.2 reduces the comparison to degree $b - 6 \geq 20$. Otherwise $d = r + 6m$ lies in the exhaustive classification of Section 5. Every pair is either cleared there by a proved certificate or the chart, or flagged. The flags have $t_2 \leq 27$ and are covered by Verification 6.1. Hence $Q(\lambda) \geq F(\lambda)$ for every source partition and every $b \geq 6$. The source has no constituents of length greater than six; specialization at extra variables zero transfers

the comparisons to every finite dimension. Complete reducibility gives the required equivariant injection. $\square$

Remark 7.1 (The independent full-range route). The same finite argument, with no change of predicate or arithmetic, continues from $b = 30$ through $b = 149$, as recorded in the full tables. Combining it with Proposition 2.7 for $b \geq 150$ proves the theorem without using the quadratic companion. The chart and certificate methods are therefore retained as a complete independent proof, while Corollary 2.1 gives the shorter large-degree route.

References

- [1] Károly J. Böröczky, Francisco Santos, and Oriol Serra. On sumsets and convex hull. *Discrete & Computational Geometry*, 52:705–729, 2014. [Author version, August 17, 2014](#). doi: [10.1007/s00454-014-9633-2](https://doi.org/10.1007/s00454-014-9633-2).
- [2] Michel Brion. Stable properties of plethysm: on two conjectures of Foulkes. *Manuscripta Mathematica*, 80:347–371, 1993. doi: [10.1007/BF03026558](https://doi.org/10.1007/BF03026558).
- [3] Michel Brion. Sur certains modules gradués associés aux produits symétriques. In *Algèbre non commutative, groupes quantiques et invariants (Reims, 1995)*, volume 2 of *Séminaires et Congrès*, pages 157–183. Société Mathématique de France, Paris, 1997. [Publisher record](#).
- [4] Peter Bürgisser, Matthias Christandl, and Christian Ikenmeyer. Even partitions in plethysms. *Journal of Algebra*, 328(1):322–329, 2011. [arXiv:1003.4474v2](#). doi: [10.1016/j.jalgebra.2010.10.031](https://doi.org/10.1016/j.jalgebra.2010.10.031).
- [5] Peter Bürgisser, Christian Ikenmeyer, and Greta Panova. No occurrence obstructions in geometric complexity theory. *Journal of the American Mathematical Society*, 32:163–193, 2019. [arXiv:1604.06431v3](#). doi: [10.1090/jams/908](https://doi.org/10.1090/jams/908).
- [6] Yue Cao, Naihuan Jing, and Ning Liu. Plethystic Murnaghan–Nakayama rule via vertex operators. *Journal of Algebraic Combinatorics*, 62:57, 2025. Article 57. doi: [10.1007/s10801-025-01480-w](https://doi.org/10.1007/s10801-025-01480-w).
- [7] Man-Wai Cheung, Christian Ikenmeyer, and Sevak Mkrtchyan. Symmetrizing tableaux and the 5th case of the Foulkes conjecture. *Journal of Symbolic Computation*, 80:833–843, 2017. [arXiv:1509.03944v1](#). doi: [10.1016/j.jsc.2016.09.002](https://doi.org/10.1016/j.jsc.2016.09.002).
- [8] Suzie C. Dent and Johannes Siemons. On a conjecture of Foulkes. *Journal of Algebra*, 226(1):236–249, 2000. doi: [10.1006/jabr.1999.8169](https://doi.org/10.1006/jabr.1999.8169).
- [9] Jacques Désarménien, Bernard Leclerc, and Jean-Yves Thibon. Hall–Littlewood functions and Kostka–Foulkes polynomials in representation theory. *Séminaire Lotharingien de Combinatoire*, 32:B32c, 1994. 38 pp. [Full text](#).
- [10] Anton Evseev, Rowena Paget, and Mark Wildon. Character deflations and a generalization of the Murnaghan–Nakayama rule. *Journal of Group Theory*, 17(6):1035–1070, 2014. doi: [10.1515/jgth-2014-0023](https://doi.org/10.1515/jgth-2014-0023).
- [11] H. O. Foulkes. Concomitants of the quintic and sextic up to degree four in the coefficients of the ground form. *Journal of the London Mathematical Society*, 25(3):205–209, 1950. doi: [10.1112/jlms/s1-25.3.205](https://doi.org/10.1112/jlms/s1-25.3.205).

- [12] William Fulton and Joe Harris. *Representation Theory: A First Course*, volume 129 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 1991.
- [13] Moritz Gangl, Álvaro Gutiérrez, and Michał Szwejk. On the generalised Foulkes conjecture for $SL_2(\mathbb{C})$ under divisibility conditions. arXiv:2507.06220v3, 2026. Version 3, July 1, 2026. doi: [10.48550/arXiv.2507.06220](https://doi.org/10.48550/arXiv.2507.06220).
- [14] Christian Ikenmeyer. On McKay’s propagation theorem for the Foulkes conjecture. arXiv:1509.04957, 2015. doi: [10.48550/arXiv.1509.04957](https://doi.org/10.48550/arXiv.1509.04957).
- [15] Kiumars Kaveh and Askold G. Khovanskii. Newton–Okounkov bodies, semigroups of integral points, graded algebras and intersection theory. *Annals of Mathematics*, 176(2):925–978, 2012. arXiv:0904.3350v3. doi: [10.4007/annals.2012.176.2.5](https://doi.org/10.4007/annals.2012.176.2.5).
- [16] J. M. Landsberg. Geometric complexity theory: an introduction for geometers. *Annali dell’Università di Ferrara*, 61(1):65–117, 2015. doi: [10.1007/s11565-014-0202-7](https://doi.org/10.1007/s11565-014-0202-7).
- [17] I. G. Macdonald. *Symmetric Functions and Hall Polynomials*. Oxford Mathematical Monographs. Clarendon Press, Oxford University Press, Oxford, second edition, 1995.
- [18] Máté Matolcsi and Imre Z. Ruzsa. Sumsets and the convex hull. In David Chudnovsky and Gregory Chudnovsky, editors, *Additive Number Theory: Festschrift in Honor of the Sixtieth Birthday of Melvyn B. Nathanson*, pages 221–227. Springer, 2010. arXiv:0810.1485v1. doi: [10.1007/978-0-387-68361-4_16](https://doi.org/10.1007/978-0-387-68361-4_16).
- [19] Tom McKay. On plethysm conjectures of Stanley and Foulkes. *Journal of Algebra*, 319(5):2050–2071, 2008. doi: [10.1016/j.jalgebra.2007.12.003](https://doi.org/10.1016/j.jalgebra.2007.12.003).
- [20] Jürgen Müller and Max Neunhöffer. Some computations regarding Foulkes’ conjecture. *Experimental Mathematics*, 14(3):277–283, 2005. doi: [10.1080/10586458.2005.10128928](https://doi.org/10.1080/10586458.2005.10128928).
- [21] OpenAI. Quadratic stabilization of the canonical Foulkes–Howe map. OpenAI Math Release preprint [OAI:Quadratic-Stabilization-of-the-Canonical-Foulkes-Howe-Map-September-25-2026](https://arxiv.org/abs/2026.09.25), 2026. Theorem 1.
- [22] Claudiu Raicu, Steven V. Sam, and Jerzy Weyman. On some modules supported in the Chow variety. *Vietnam Journal of Mathematics*, 50(2):501–521, 2022. arXiv:2108.10910v1; locators refer to this preprint. doi: [10.1007/s10013-021-00527-2](https://doi.org/10.1007/s10013-021-00527-2).
- [23] Claudiu Raicu, Steven V. Sam, Jerzy Weyman, and Fuxiang Yang. Powers of binary forms and derived Hermite reciprocity. arXiv:2602.15175v2, 2026. Version 2, September 2, 2026. doi: [10.48550/arXiv.2602.15175](https://doi.org/10.48550/arXiv.2602.15175).
- [24] David Rydh. A minimal set of generators for the ring of multisymmetric functions. *Annales de l’Institut Fourier*, 57(6):1741–1769, 2007. doi: [10.5802/aif.2312](https://doi.org/10.5802/aif.2312).
- [25] R. M. Thrall. On symmetrized Kronecker powers and the structure of the free Lie ring. *American Journal of Mathematics*, 64:371–388, 1942. doi: [10.2307/2371691](https://doi.org/10.2307/2371691).
- [26] Francesco Vaccarino. The ring of multisymmetric functions. *Annales de l’Institut Fourier*, 55(3):717–731, 2005. doi: [10.5802/aif.2111](https://doi.org/10.5802/aif.2111).
- [27] Mark Wildon. A combinatorial proof of a plethystic Murnaghan–Nakayama rule. *SIAM Journal on Discrete Mathematics*, 30(3):1526–1533, 2016. arXiv:1408.3554v2. doi: [10.1137/14098260X](https://doi.org/10.1137/14098260X).

A The bounded certificate program

The following is the complete first program. It uses GNU C++17 signed 128-bit integers and optional OpenMP; assertions must remain enabled. The source file is `verification/programs/appendix-a.cpp`. See Sections 3–5 for the construction and Section 7 for arithmetic and reproduction.

```
1
2 // Compile assertions enabled, e.g. g++ -std=c++17 -O3 -march=native -fopenmp v.cpp
3 // OpenMP optional; set a moderate OMP_NUM_THREADS (e.g. 4)
4 #include <memory>
5 #include <cstdlib>
6 #include <utility>
7 #include <iostream>
8 #include <vector>
9 #include <array>
10 #include <algorithm>
11 #include <cstdint>
12 #include <chrono>
13 #include <cstring>
14 #include <cassert>
15 #include <iomanip>
16 #include <numeric>
17 #include <map>
18 using namespace std;
19 using i64=int64_t;
20 using i128=__int128_t;
21 ostream& operator<<(ostream& o, const i128 a) {
22     if (a<0) return o<<'-'<<-a;
23     if (a>9) o<<a/10;
24     return o<<int(a%10);
25 }
26 const int K=6, M=200*K+50, D=6;
27 using Index=array<int,D>; // descending parts (+ staircase shift when needed)
28 i64 rank_tab[D+1][M][M];
29 void init() {
30     // rank_tab[n][w][k] # partitions of w length<=n with smallest of n <k
31     rank_tab[0][0][M-1]=1;
32     for(int d=1;d<=D;d++)
33         for(int w=0;w<M;w++) for(int k=1,t=w;k<M;k++,t-=d)
34             rank_tab[d][w][k]=rank_tab[d][w][k-1]+(t<0?0:rank_tab[d-1][t][M-1]);
35 }
36 template<bool shifted=false>
37 int rankp(const Index& l, int w) {
38     int idx=0, prev=0;
39     for(int j=D-1;j>=1;j--) {
40         int p=l[j]-(shifted?(D-1-j):0);
41         auto t=rank_tab[j+1][w];
42         idx+=t[p]-t[prev];
43         w-=p; prev=p;
44     }
45     return idx;
46 }
47 Index shift(Index l) {
48     for(int i=0;i<D;i++)l[i]+=D-1-i;
49     return l;
50 }
51 int straight(Index& l) { // descending with sign
52     int inv=false;
53     for(int i=1;i<D;i++) {
54         int x=l[i], j=i;
55         while(j>0&&x>l[j-1]){l[j]=l[j-1]; inv=!inv; j--;}
56         if (j>0&&x==l[j-1])return 0;
57         l[j]=x;
58     }
59     return inv?-1:1;
60 }
61 i128 get(const vector<i128>& tab, int b, Index l) {
```

```

62 int s=straight(l); if(!s || l.back()<0)return 0;
63 int ix=rankp<true>(l,b*K);
64 if (s<0)return -tab[ix];
65 return tab[ix];
66 }
67
68 template<class F>
69 void comp_impl(int w, Index& l, int j, F f) {
70   if(j==D-1){l[j]=w;f(l);return;}
71   for(int x=0;x<=w;x++){
72     l[j]=x;
73     comp_impl(w-x,l,j+1,f);
74   }
75 }
76 template<class F>
77 void comps(int w, F f) {Index l;comp_impl(w,l,0,f);}
78 template<class F>
79 void part_impl(int w, Index& l, int j, F f) {
80   if(j==0){l[j]=w;f(l);return;}
81   for(int x=(j==D-1?0:l[j+1]);x*(j+1)<=w;x++){
82     l[j]=x;
83     part_impl(w-x,l,j-1,f);
84   }
85 }
86 template<class F>
87 void parts(int w, F f) {Index l;part_impl(w,l,D-1,f);}
88 vector<vector<i128>> gen(int B) {
89   vector<vector<Index>> sub(B+1);
90   for(int i=1;i<=B;i++) {
91     comps(K,[&](Index l){for(int& x:l)x*=i;sub[i].push_back(l);});
92   }
93   vector<vector<i128>> tabs(B+1);
94   tabs[0]={1};
95   #pragma omp parallel
96   for(int b=1;b<=B;b++){
97
98     int N=rank_tab[D][b*K][M-1];
99     #pragma omp single
100     tabs[b].resize(N);
101
102     vector<Index> ps;ps.reserve(N);
103     parts(K*b,[&](Index l){ps.push_back(shift(l));});
104     #pragma omp for schedule(dynamic,10)
105     for(int j=0;j<N;j++){
106       const Index& l=ps[j];i128 t=0;
107       for(int i=1;i<=b;i++) for(auto& delta:sub[i]){
108         Index prev; bool ok=true;
109         for(int j=0;j<D;j++)if((prev[j]=l[j]-delta[j])<0){ok=false;break;}
110         if(ok && __builtin_add_overflow(t,get(tabs[b-i],b-i,prev),&t))abort();
111       }
112       if(t<0||t%b){std::cerr<<"bad\n";abort();}
113       tabs[b][j]=t/b;
114     }
115   }
116 }
117 return tabs;
118 }
119
120
121 // shorter-length table using the same target recurrence
122 template<int n>
123 struct Make {
124   vector<vector<i128>> tab{vector<i128>{1}};
125   vector<Index> ps;
126   vector<vector<Index>> subs;
127   int rankn(Index l, int w) const {
128     int idx=0, prev=0;
129     for(int j=n-1;j>=1;j--) {

```

```

130     int p=l[j]-(n-1-j);
131     auto t=rank_tab[j+1][w];idx+=t[p]-t[prev];
132     w-=p;prev=p;
133 }
134 return idx;
135 }
136 Make(int B):subs(B+1) {
137     comps(6,[&](Index d){
138         for(int i=n;i<6;i++)if(d[i])return;
139         for(int j=1;j<=B;j++){Index l=d;for(int i=0;i<n;i++)l[i]*=j;subs[j].push_back(1);}
140     });
141 }
142 i128 at(Index l,int b) const {
143     // l shifted by (n-1,..0)
144     int inv=false;
145     for(int i=1;i<n;i++){
146         int x=l[i], j=i;
147         while(j>0&&x>l[j-1]){l[j]=l[j-1];inv=!inv;j--;}
148         if(j&&x==l[j-1])return 0;
149         l[j]=x;
150     }
151     auto t=tab[b][rankn(l,b*6)];
152     return inv?-t:t;
153 }
154 static void sub_shift(Index& l){ for(int i=0;i<n;i++)l[i]+=n-1-i; }
155 void next(int b) {
156     int N=rank_tab[n][6*b][M-1];
157     ps.clear();ps.reserve(N);
158     Index zero{};part_impl(6*b,zero,n-1,[&](Index l){sub_shift(l);ps.push_back(1);});
159     tab.emplace_back(ps.size());if(ps.size()!=N)abort();
160     auto& q=tab.back();
161     #pragma omp parallel for schedule(dynamic,200)
162     for(int k=0;k<N;k++){
163         Index p=ps[k];i128 c=0;bool over=false;
164         for(int i=1;i<=b;i++)for(auto& l:subs[i]){
165             Index t{};
166             for(int j=0;j<n;j++){t[j]=p[j]-l[j];if(t[j]<0) goto nope;}
167             over|=__builtin_add_overflow(at(t,b-i),c,&c);
168             nope:;
169         }
170         if(over){cerr<<"overflow at b="<<b;abort();}
171         if(c<0||c%b){cerr<<"bad q";abort();}
172         q[k]=c/b;
173     }
174 }
175 };
176
177 template<int n>
178 struct TailRank {
179     int T,w;
180     vector<int> off;
181     vector<Index> ts;
182     static int ntails(int W){
183         int n0=6-n,q=1;
184         for(int i=0;i<n0;i++)q*=W+1;
185         return q;
186     }
187     int tindex(const Index& l) const{
188         int i=0;
189         for(int j=n;j<6;j++)i=i*(T+1)+l[j];
190         return i;
191     }
192     int index(const Index& l) const{
193         int k=tindex(l);
194         int f=l[n],z=w;
195         for(int j=n;j<6;j++)z-=l[j];
196         z-=f*n;
197         int idx=off[k], prev=0;

```

```

198 for(int j=n-1;j>=1;j--){
199     int p=l[j]-f;
200     auto t=rank_tab[j+1][z];idx+=t[p]-t[prev];
201     z-=p;prev=p;
202 }
203 return idx;
204 }
205 const int size() const {return off.back();}
206 TailRank(int w0,int t0):w(w0),T(t0) {
207     off.resize(ntails(T)+1,-1);int idx=0;Index p{};
208     auto rec=[&](auto rec,int j,int z)->void{
209         if(j>=n){
210             int f=j==5?0:p[j+1];
211             for(int k=f;k*(j+1-n)<=T-z && k*(j+1)<=w-z;k++){p[j]=k;rec(rec,j-1,z+k);}
212         } else {
213             int k=tindex(p);off[k]=idx;ts.push_back(p);
214             idx+=rank_tab[n][w-z-n*p[n]][M-1];
215         }
216     };
217     rec(rec,5,0);
218     off.back()=idx;
219 }
220 vector<Index> parts(){
221     vector<Index> out;out.reserve(size());
222     for(Index p:ts){
223         int z=w;for(int j=n;j<6;j++)z-=p[j];
224         part_impl(z,p,n-1,[&](Index l){out.push_back(l)});
225     }
226     if(out.size()!=size()){cerr<<"size bug\n";abort();}
227     return out;
228 }
229 };
230
231 template<int n>
232 struct QTail {
233     int T;vector<TailRank<n>> ranks;
234     vector<vector<i128>> tab;
235     QTail(int maxb,int T):T(T) {
236         ranks.reserve(maxb+1);
237         for(int b=0;b<=maxb;b++)ranks.emplace_back(b*6,T);
238     }
239     int B() const {return ranks.size()-1;}
240     i128 get(Index l, int b) const{ // nonnegative shifted vector, not necessarily strict/sorted
241         int s=straight(l);if(!s)return 0;
242         for(int i=0;i<6;i++)l[i]-=5-i;
243         return s*tab[b][ranks[b].index(l)];
244     }
245
246     void next(int b){
247         auto& rank=ranks[b];auto ps=rank.parts();
248         tab.emplace_back(ps.size());auto& q=tab.back();
249
250         #pragma omp parallel for schedule(dynamic,100)
251         for(int k=0;k<ps.size();k++){
252             Index p=shift(ps[k]);i128 c=0;bool over=false;
253             for(int i=1;i<=b;i++){
254                 auto rec=[&](auto rec,Index& t,int j,int z)->void{
255                     if(z==0){over|=__builtin_add_overflow(get(t,b-i),c,&c);}
256                     else if(j==0){
257                         t[0]-=z;
258                         if(t[0]>=0)over|=__builtin_add_overflow(get(t,b-i),c,&c);
259                         t[0]+=z;
260                     } else {
261                         int q=t[j],v=max(q-z,0);
262                         for(;t[j]>=v;t[j]-=i)rec(rec,t,j-1,z+t[j]-q);
263                         t[j]=q;
264                     }
265                 };

```

```

266     rec(rec,p,5,6*i);
267 }
268 if(over){cerr<<"overflow "<<b<<endl;abort();}
269 if(c<0||c%b){cerr<<"bad qtail";abort();}
270 q[k]=c/b;
271 }
272 }
273 i128 coeff(Index l, int b){
274     int t=0;for(int j=n;j<6;j++)t+=l[j];
275     if(t>T || b>B())return 0;
276     return tab[b][ranks[b].index(l)];
277 }
278 };
279
280 template<class F>
281 void strip(Index l, int n, int i, int b, F f) {
282     // iterate contribution to p_i[h_b]*something(length<=n-i),  $\lambda=1$  length<=n, prior weight (n-i)b.
283     int last[6],idx[6],width[6], m=0;
284     Index p=l,mu{};
285     for(int j=0;j<n;j++)p[j]+=n-1-j;
286     for(int j=0;j<i;j++)last[j]=-1;
287     int rem=b, sum=0;
288     for(int j=n-1;j>=0;j--){
289         int r=p[j]%i,k=last[r];
290         if(k==--1){
291             mu[j]=r;rem--=(p[j]-r)/i;
292         }else{
293             idx[m]=j;width[m]=(p[j]-p[k])/i-1;
294             sum+=width[m];++m;
295         }
296         last[r]=j;
297     }
298     if(m!=n-i || rem<0 || sum<rem)return; // m mismatch => some residues not present
299
300     auto call=[&]() {
301         Index s=mu;
302         int sign=1;
303         for(int j=1;j<n;j++){
304             int x=s[j],k=j;
305             while(k&&x>s[k-1]){s[k]=s[k-1];k--;sign=-sign;}
306             s[k]=x;
307         }
308         for(int j=0;j<n;j++)s[j]-=n-1-j;
309         f(s,sign);
310     };
311     auto rec=[&](auto rec,int k,int w,int max)->void{
312         if(k==m){call();return;}
313         int j=idx[k];max-=width[k];
314         for(int x=std::max(0,w-max);x<=w&&x<=width[k];x++){
315             mu[j]=p[j]-i*x;
316             rec(rec,k+1,w-x,max);
317         }
318     };
319     rec(rec,0,rem,sum);
320 }
321
322 struct Source {
323     int b;vector<vector<i128>> data{vector<i128>{1}};
324     Source(int b):b(b){}
325     i128 next(Index l,int j) const{
326         i128 c=0;
327         for(int i=1;i<=j;i++) strip(l,j,i,b,[&](Index mu,int s){
328             c+=s*data[j-i][rankp(mu,(j-i)*b)];
329         });
330         if(c<0||c%j){cerr<<"bad source";abort();}
331         return c/j;
332     }
333     void grow(int j){

```

```

334 auto w=j*b,N=int(rank_tab[j][w][M-1]);
335 vector<Index> ps;ps.reserve(N);Index p{};
336 part_impl(w,p,j-1,[&](Index l){ps.push_back(l);});
337 data.emplace_back(ps.size());auto& q=data.back();
338 #pragma omp parallel for schedule(dynamic,50)
339 for(int k=0;k<N;k++)q[k]=next(ps[k],j);
340 }
341 };
342
343
344 i128 binom(int n,int k){
345     i128 c=1; k=min(k,n-k);
346     for(int i=1;i<=k;i++) c=c*(n-i+1)/i;
347     return c;
348 }
349 constexpr i128 INF=i128(1)<<100;
350 i128 kbound(i128 p,int b,int k) {
351     if(p==0)return 0;
352     p=min(p,INF);
353     int z=0;
354     while(binom(b+z,z)<p) ++z;
355     if(z>18)z=18; // avoid overflow choose(167,18)~1e23; INF checks
356     auto c=binom(k+z,z), h=binom(k+z-1,z);
357     if (p-1-z>INF/h)return INF;
358     return min(INF,c+(p-1-z)*h);
359 }
360 int deg(const Index& r) {
361     int w=0;
362     for(int j=0;j<D;j++)w+=(j+1)*r[j];
363     return w;
364 }
365 Index add(const Index& a, const Index& b) {
366     auto c=a;
367     for(int j=0;j<D;j++)c[j]+=b[j];
368     return c;
369 }
370 Index mul(Index a,int k){
371     for(auto& x:a)x*=k;
372     return a;
373 }
374 int tail(const Index& d,int n=2) {
375     int t=0;
376     for(int j=n;j<D;j++) t+=(j+1-n)*d[j];
377     return t;
378 }
379 Index part(Index d){
380     for(int j=D-2;j>=0;j--)d[j]+=d[j+1];
381     return d;
382 }
383 vector<vector<i128>> data_;
384 int const cap=25,maxb=149;
385 i128 coeff(const Index& d){
386     int w=deg(d);
387     assert(w%6==0);
388     if(w>cap*6)return 0;
389     return data_[w/K][rankp(part(d),w)];
390 }
391 template<class F>
392 void residues(Index r, int j, F f){
393     for(int x=0;x<6;x++){
394         r[j]=x;
395         if(j==0) {
396             if(deg(r)%6==0) f(r);
397         } else residues(r,j-1,f);
398     }
399 }
400
401 struct cert { Index l; i128 val; };

```

```

402
403 Index diff(Index a, const Index& b){for(int i=0;i<D;i++)a[i]-=b[i];return a;}
404
405 struct Grid {
406     int limit;
407     vector<Index> points,prev;
408     vector<i64> tot;
409     Grid(int lim):limit(lim),tot(lim+2) {
410         for(int w=0;w<=limit;w++){
411             tot[w+1]=tot[w]+rank_tab[5][w][M-1];
412         }
413         points.reserve(tot.back());
414         prev.reserve(tot.back());
415         for(int w=0;w<=limit;w++){
416             Index d{},p{};
417             part_impl(w,p,4,[&](Index l){
418                 d[4]=l[4];
419                 for(int i=0;i<4;i++)d[i]=l[i]-l[i+1];
420                 points.push_back(d);
421                 for(int i=0;i<=4;i++) {
422                     --d[i];
423                     l[i]=index(d);
424                     ++d[i];
425                 }
426                 prev.push_back(l);
427             });
428             if(points.size()!=tot[w+1])abort();
429         }
430     }
431     int index(const Index& d) const {
432         for(int v:d)if(v<0)return -1;
433         if(d[5])return -1;
434         int w=deg(d); if(w>limit)return -1;
435         return tot[w]+rankp(part(d),w);
436     }
437     size_t size(int max=-1) const {return max<0?points.size():tot[min(limit,max)+1];}
438 };
439 Grid *g;
440
441 struct Mono {
442     int n;
443     vector<i128> q;
444     template<class F> void exact(int max,F f){
445         n=g->size(max);q.resize(n);
446         for(int i=0;i<n;i++)q[i]=f(g->points[i]);
447     }
448     void propagate(){
449         for(int i=0;i<n;i++){
450             for(int j=0;j<5;j++){
451                 int p=g->prev[i][j];if(p<0)continue;
452                 q[i]=max(q[i],q[p]);
453             }
454         }
455     }
456     void extend(int max) {n=g->size(max);q.resize(n);propagate();}
457     i128 at(const Index& d) const {int i=g->index(d);return i>=0&&i<n?q[i]:0;}
458 };
459 Mono powerAll;
460 vector<vector<int>>> masksList;
461 vector<int> maskCounts;
462 void init_masks(){
463     for(int k=1;k<=6;k++){
464         vector<int> p(k);
465         auto rec=[&](auto rec,int pos,int left,int lo)->void{
466             if(pos==k){
467                 if(left)return;
468                 i64 c=720;vector<int> t(7);
469                 for(int i:p)c=c/(++t[i]*i);

```



```

470     vector<int> masks;
471     do{
472         int s=6,m=0;
473         for(int i:p) {
474             s-=i;
475             if(s>=2)m|=1<<(s-2);
476         }
477         if(find(masks.begin(),masks.end(),m)==masks.end())masks.push_back(m);
478     } while(next_permutation(p.begin(),p.end()));
479     masksList.push_back(masks);
480     maskCounts.push_back(c);
481     return;
482 }
483 for(int i=lo;i<=left;i++){p[pos]=i;rec(rec,pos+1,left-i,i);}
484 };
485 rec(rec,0,6,1);
486 }
487 if(accumulate(maskCounts.begin(),maskCounts.end(),0)!=720){cerr<<"counts fail: ";for(auto v:maskCounts)cerr<<
    v<<" ";cerr<<endl;abort();}
488 }
489 i128 ub_cycle(const Index& d){
490     i128 q[16];
491     q[0]=1;
492     for(int i=0;i<4;i++){
493         int s=i+2;int w[5],best=0;
494         for(int j=0;j<s;j++){
495             w[j]=1;
496             for(int k=j;k<j+6-s;k++)w[j]+=d[k];
497             if(w[j]>w[best])best=j;
498         }
499         i128 f=1;
500         for(int j=0;j<s;j++)if(j!=best)f*=w[j];
501         for(int m=0;m<1<<i;m++)q[m+(1<<i)]=q[m]*f;
502     }
503     i128 u=0;
504     for(int j=0;j<masksList.size();j++){
505         i128 t=INF;
506         for(int m:masksList[j])t=min(t,q[m]);
507         u+=i128(maskCounts[j])*t;
508     }
509     return u/720;
510 }
511 void feed_power(Index x,i128 p){
512     // x[5]==0 required; other shifts via residues separately
513     if(p<=1||x[5])return;
514     int b=deg(x)/6;
515     for(int k=1;k*b<=maxb;k++){
516         auto d=mul(x,k);
517         bool ok=true;
518         for(int i=0;i<5;i++){
519             if(d[i]%6){ok=false;break;}
520             d[i]/=6;
521         }
522         if(!ok)continue;
523         int w=g->index(d);
524         powerAll.q[w]=max(powerAll.q[w],kbound(p,b,k));
525     }
526 }
527 vector<Index> rs;
528 const int ulim=42;
529 Mono uniBase;
530 vector<cert> uOcc;
531 void init_common(){
532     g=new Grid(maxb);
533     residues(Index{},D-1,[&](Index r){rs.push_back(r)});
534     if(rs.size()!=7776)abort();
535     powerAll.exact(maxb,[](Index x){return i128(1)});
536     uniBase.exact(ulim,[](Index x){return INF});

```

```

537 Mono specific;
538 for(Index r:rs){
539     int b0=deg(r)/6;
540     specific.exact(cap-b0,[&](Index m){
541         return coeff(add(r,mul(m,6)));
542     });
543     if(r[5]==0)for(int i=0;i<specific.n;i++) feed_power(add(r,mul(g->points[i],6)),specific.q[i]);
544     specific.extend(ulim);
545     for(int i=0;i<specific.n;i++)uniBase.q[i]=min(uniBase.q[i],specific.q[i]);
546 }
547 // minimal common occurrences
548 for(int i=0;i<uniBase.n;i++){
549     if(!uniBase.q[i])continue;
550     bool minimal=true;
551     for(int j=0;j<5;j++){
552         int p=g->prev[i][j];
553         if(p>=0&&uniBase.q[p])minimal=false;
554     }
555     if(minimal)uOcc.push_back(cert{g->points[i],uniBase.q[i]});
556 }
557 cerr<<uOcc.size()<<" minimal common occurrences with all r from deg<= "<<ulim<<endl;
558 uniBase.extend(maxb);
559 powerAll.propagate();
560 }
561 vector<vector<i128>> qextra[6];
562 int xcap[6]={0,0,0,149,55,25};
563 template<int n> void gen_extra(){
564     Make<n> make(xcap[n]);
565     for(int b=1;b<=xcap[n];b++)make.next(b);
566     qextra[n]=move(make.tab);
567 }
568 void feed_extra(int n){
569     for(int b=cap+1;b<=xcap[n];b++){
570         Index d{},p{};int i=0;
571         part_impl(b*6,p,n-1,[&](Index l){
572             auto val=qextra[n][b][i++];
573             if(n>3&&l[n-1]==0)return;
574             d[n-1]=l[n-1];
575             for(int j=0;j<n-1;j++)d[j]=l[j]-l[j+1];
576             if(val>INF)val=INF;
577             feed_power(d,val);
578         });
579     }
580     powerAll.propagate();
581 }
582 i128 explicit_coeff(Index d){
583     int w=deg(d),b=w/6,n=6;
584     if(b<=cap)return coeff(d);
585     while(n>4&&!d[n-1])n--;
586     if(n>=5 || b>xcap[n] || w!=6*b)return 0;
587     return qextra[n][b][rankp(part(d),w)];
588 }
589
590 Index high{5,5,5,5,5,5};
591 // singleton common lower test
592 bool passed(Index m, Index dhi){
593     i128 u=ub_cycle(dhi),v=uniBase.at(m);
594     if(v>=u)return true;
595     for(auto z:uOcc){
596         auto p=diff(m,z.l);
597         v=max(v,powerAll.at(p));
598     }
599     return v>=u;
600 }
601
602 unique_ptr<QTail<3>> qt;
603 i64 fails_by[25] {}, maxby[25] {};
604 vector<int> todos;

```

```

605 vector<int> down, prevIndex, where;
606 vector<Index> downprev;
607 void setup(){
608     where.resize(g->size(),-1);
609     vector<int> todo1;
610     for(int i=0;i<g->size();i++){
611         auto m=g->points[i];
612         auto d=mul(m,6), dhi=add(d,high);
613         int b=deg(m);
614         if(b+17<=cap || passed(m,dhi))continue;
615         where[i]=0;todo1.push_back(i);
616     }
617     for(int i=g->size()-1;i>=0;i--){
618         if(where[i]==-1)continue;
619         for(int j=0;j<5;j++){
620             int p=g->prev[i][j];if(p>=0)where[p]=0;
621         }
622     }
623     for(int i=0;i<g->size();i++){
624         if(where[i]==-1)continue;
625         Index p=g->prev[i];
626         for(int j=0;j<5;j++)if(p[j]>=0)p[j]=where[p[j]];
627         where[i]=down.size();
628         downprev.push_back(p);
629         down.push_back(i);
630     }
631     for(int p:todo1)todos.push_back(where[p]);
632     cerr<<"todos "<<todos.size()<<"", down "<<down.size()<<endl;
633 }
634
635
636 vector<vector<Index>> occvec;
637
638 void init_occ(){
639     occvec.resize(rs.size());
640     for(int i=0;i<rs.size();i++){
641         auto r=rs[i];Mono spec;
642         spec.exact(cap-deg(r)/6, [&](Index m){return coeff(add(r,mul(m,6)));});
643         for(int z=0;z<spec.n;z++){
644             if(!spec.q[z])continue;
645             bool minimal=true;
646             for(int j=0;j<5;j++){
647                 int v=g->prev[z][j];if(v>=0&&spec.q[v])minimal=false;
648             }
649             if(minimal)occvec[i].push_back(g->points[z]);
650         }
651     }
652     int m=0;for(auto& c:occvec)m=max(m,int(c.size()));
653     cerr<<"max occvec "<<m<<endl;
654 }
655
656 struct TestSpec {
657     Index r;
658     vector<i128> spec;
659     vector<Index> occ;
660     TestSpec(int i):r(rs[i]),occ(occvec[i]) {
661         spec.reserve(down.size());
662         for(int k=0;k<down.size();k++){
663             auto& m=g->points[down[k]];
664             auto d=add(r,mul(m,6));
665             i128 q=max(explicit_coeff(d),qt->coeff(part(d),deg(d)/6));
666             for(int j=0;j<5;j++){
667                 int p=downprev[k][j];if(p>=0)q=max(q,spec[p]);
668             }
669             spec.push_back(q);
670         }
671     }
672     bool pass(const Index& m,const Index& d,i128 q) const {

```

```

673 auto u=ub_cycle(d);
674 if(q>u)return true;
675 // direct chart: non-first tail <=b suffices
676 if(tail(d,1)<=deg(d)/6)return true;
677 for(auto& c:occ)if(powerAll.at(diff(m,c))>=u)return true;
678 return false;
679 }
680 void run() const{
681     for(int k:todos){
682         auto& m=g->points[down[k]];
683         auto d=add(r,mul(m,6));
684         int b=deg(d)/6;
685         if(b<=cap||b>maxb)continue;
686         if(pass(m,d,spec[k]))continue;
687
688         #pragma omp critical
689         {
690             ++fails_by[b/6];maxby[b/6]=max(maxby[b/6],i64(tail(d)));
691         }
692     }
693 }
694 };
695 void classify(){
696     init_common();
697     gen_extra<3>(); gen_extra<4>(); feed_extra(3);feed_extra(4);
698     setup();init_occ();
699     qt=make_unique<QTail<3>>(45,18);qt->tab={{1}};
700     for(int b=1;b<=45;b++)qt->next(b);
701     #pragma omp parallel for schedule(dynamic,4)
702     for(int i=0;i<rs.size();i++){TestSpec t(i);t.run();}
703     for(int i=cap/6;i<25;i++){
704         cout<<6*i<<" flags "<<fails_by[i]<<" max "<<maxby[i]<<endl;
705         if(maxby[i]>27)abort();
706     }
707 }
708
709 i64 checked=0, mistakes=0;
710 void run(int b){
711     Source s(b);for(int j=1;j<=5;j++)s.grow(j);
712     i64 c=0,eq=0;
713     vector<Index> todo;
714     parts(6*b,[&](Index l){
715         Index d=l;for(int j=0;j<5;j++)d[j]-=l[j+1];
716         if(coeff(d)<ub_cycle(d))todo.push_back(l);
717     });
718     #pragma omp parallel for schedule(dynamic,25) reduction(+:c,eq)
719     for(int k=0;k<todo.size();k++){
720         Index l=todo[k],d=l;for(int j=0;j<5;j++)d[j]-=l[j+1];
721         auto p=s.next(l,6), q=coeff(d);
722         if(p>q){
723             c++;
724             #pragma omp critical
725             if(mistakes++<10){cerr<<"Violation b="<<b<<" l=";for(int v:l)cerr<<v<<" ";cerr<<" f="<<p<<" q="<<q<<endl;}
726         }
727         if(p==q)eq++;
728     }
729     checked+=todo.size();
730     cout<<b<<" total(exact) "<<todo.size()<<" failed "<<c<<" equality "<<eq<<endl;
731 }
732
733 int main(){
734     init();init_masks();data_=gen(cap);
735     for(int b=6;b<=cap;b++)run(b);
736     if(mistakes)return 1;
737     classify();
738 }

```

B The tail-band program

The following is the complete second program. It uses GNU C++17 and Boost.Multiprecision for the final coefficient sums. The source file is `verification/programs/appendix-b.cpp`. See Section 6 for the construction and Section 7 for arithmetic and reproduction.

```
1 #include <boost/multiprecision/cpp_int.hpp>
2 #include <stdexcept>
3 #include <functional>
4 #include <cstdint>
5 #include <iostream>
6 #include <algorithm>
7 #include <array>
8 #include <vector>
9 #include <cassert>
10 using namespace std;
11 using Int=__int128_t;
12 using Big=boost::multiprecision::cpp_int;
13 using A=array<int,4>;
14 const int T=27,O=168,L=330,NB=447+O+1;
15 using P=array<Int,L>; // q exponents -168..161
16 vector<A> parts;
17 int index_[28][28][28][28];
18 int idx(A p,int sign=0){
19     sort(p.begin(),p.end(),greater<int>());
20     if(p.back()<0)return -1;
21     return index_[p[0]][p[1]][p[2]][p[3]];
22 }
23 int wt(A p){return p[0]+p[1]+p[2]+p[3];}
24 A sub(A p,A a,int k=1){for(int i=0;i<4;i++)p[i]-=k*a[i];return p;}
25
26 template<class F> void arrays(int t,bool ordered,F f){
27     A p{};
28     auto rec=[&](auto rec,int i,int left,int bound)->void{
29         if(i==4){if(!left)f(p);return;}
30         for(int x=0;x<=left&& x<=bound;x++){
31             p[i]=x;rec(rec,i+1,left-x,ordered?x:t);
32         }
33     };
34     rec(rec,0,t,t);
35 }
36 vector<P> num[7];
37 void add(P& to,const P& from,int mul,int shift){
38     for(int i=max(0,-shift);i<min(L,L-shift);i++)to[i+shift]+=mul*from[i];
39 }
40 void target(){
41     for(int j=0;j<=6;j++){
42         vector<P> H(parts.size());H[0][0]=1;
43         for(int k=1;k<parts.size();k++){
44             auto p=parts[k];int t=wt(p);
45             for(int h=1;h<=6&&h<=t;h++)arrays(h,false,[&](A a){
46                 for(int i=1;i*h<=t;i++){
47                     int z=idx(sub(p,a,i)); if(z<0)break;
48                     for(int u=0;u<=6-h;u++)add(H[k],H[z],h,i*(u-j));
49                 }
50             });
51             for(auto& v:H[k]){assert(v%t==0);v/=t;}
52         }
53     }
54     for(int mask=0;mask<64;mask++){if(__builtin_popcount((unsigned)mask)==j){
55         int e=0;for(int i=0;i<6;i++){if(mask>>i&1)e+=i+1;
56         for(int k=0;k<parts.size();k++)add(num[j][k],H[k],j%2?-720:720,e);
57     }
58 }
59 void source(const vector<int>& lens,int cnt){
60     int D[22]={1};
61     for(int i=1;i<=6;i++)for(int n=21;n>=i;n--)D[n]-=D[n-i];
```

```

62 for(int i:lens)for(int n=i;n<=21;n++)D[n]+=D[n-i];
63 for(int mask=0;mask<1<<lens.size();mask++){
64     int ways[2][T+1]{};ways[0][0]=ways[1][0]=1;
65     int j=0,c=cnt;
66     for(int k=0;k<(int)lens.size();k++){
67         int i=lens[k],z=mask>>k&1;
68         if(z)j+=i,c=-c;
69         for(int n=i;n<=T;n++)ways[z][n]+=ways[z][n-i];
70     }
71     for(int k=0;k<(int)parts.size();k++){
72         array<Int,T+1> pol{};pol[0]=1;int deg=0;
73         for(int m:parts[k]){
74             array<Int,T+1> t{};
75             for(int n=0;n<=deg;n++)for(int i=0;i<=m;i++)
76                 t[n+i]+=pol[n]*ways[1][i]*ways[0][m-i];
77             pol=t;deg+=m;
78         }
79         for(int i=0;i<=deg;i++)for(int e=0;e<=15;e++)
80             num[j][k][0+j+e-i]-=c*pol[i]*D[e];
81     }
82 }
83 }
84 void gen_source(vector<int>& lens,int rem,int low,int cnt){
85     if(!rem){source(lens,cnt);return;}
86     for(int h=low;h<=rem;h++){
87         int m=count(lens.begin(),lens.end(),h)+1;
88         lens.push_back(h);gen_source(lens,rem-h,h,cnt/(h*m));lens.pop_back();
89     }
90 }
91 const int pascal[5][5]={1},{1,1},{1,2,1},{1,3,3,1},{1,4,6,4,1};
92 void cross(){
93     for(int k=(int)parts.size()-1;k>=0;k--){
94         A p=parts[k];
95         for(int mask=1;mask<81;mask++){
96             int t=mask,n1=0,n2=0;A h{};
97             for(int& v:h){v=t%3;t/=3;n1+=v==1;n2+=v==2;}
98             int i=idx(sub(p,h));if(i<0)continue;
99             for(int j=0;j<=6;j++)for(int s=0;s<=n1;s++)add(num[j][k],num[j][i],(n1%2?-1:1)*pascal[n1][s],-n2-s);
100         }
101         for(int j=0;j<=6;j++)for(int s=L-1;s>0;s--)num[j][k][s]-=num[j][k][s-1];
102     }
103 }
104 using Row=array<Big,NB>;
105 Row schur_row(const vector<P>& n,A eta){
106     A delta{3,2,1,0}, p;for(int i=0;i<4;i++)p[i]=eta[i]+delta[i];
107     P r{};
108     do{
109         int i=idx(sub(p,delta));if(i<0)continue;
110         int c=1;
111         for(int v=0;v<4;v++)for(int w=0;w<v;w++)if(delta[w]<delta[v])c=-c;
112         add(r,n[i],c,0);
113     }while(prev_permutation(delta.begin(),delta.end()));
114     Row t{};for(int i=0;i<L;i++)t[i]=r[i];
115     for(int h=1;h<=6;h++)for(int i=h;i<NB;i++)t[i]+=t[i-h];
116     return t;
117 }
118 int total[25] {},eq_[25] {};
119 void run_thin(){
120     parts.clear(); // run once
121     for(int t=0;t<=T;t++)arrays(t,true,[&] (A p){index_[p[0]][p[1]][p[2]][p[3]]=parts.size();parts.push_back(p);})
122     ;
123     for(auto& v:num)v.resize(parts.size());
124     target();vector<int> lens;gen_source(lens,6,1,720);cross();
125     for(auto p:parts){
126         array<Row,7> rows;for(int j=0;j<=6;j++)rows[j]=schur_row(num[j],p);
127         for(int b=26;b<=149;b++)for(int B=p[0];2*B<=6*b-wt(p);B++){
128             Big c=0;
129             for(int j=0;j<=6;j++){int i=0+B-j*b;if(i>=0)c+=rows[j][i];}

```

```

129     if(c<0||c%720!=0)throw std::runtime_error("thin");
130     eq_[b/6]+= c==0; total[b/6]++;
131 }
132 }
133 for(int b=4;b<25;b++)cout<<6*b<<" "<<total[b]<<" "<<eq_[b]<<endl;
134 }
135 int main(){run_thin();}

```